HAW Kiel
Hochschule für Angewandte Wissenschaften Kiel
Kiel University of Applied Sciences

Master Thesis

Evaluation of Agentic AI models for Optimizing Multi-Energy system scheduling

Degree: Master of Science Data Science

Author: Socrates Gebremedhin Gizaw
Matriculation Number: 946176
First Supervisor: Prof. Dr. Stephan Doerfel
Second Supervisor: Christian Raddan
Submission Date: March 13,2026

Declaration of Authorship

I, Socrates Gebmremedhin Gizaw, declare that this thesis titled Evaluation of Agentic AI models for Optimizing Multi-Energy system scheduling, and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by me jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: _____ Location: _____

Socrates Gebremedhin Gizaw Date: 13, March 2026

Acknowledgements

I am grateful to have worked on the thesis project 'Evaluation of Agentic AI models for Optimizing Multi-Energy system scheduling' application, as it provided insights into efficient energy system production and how ML and AI can fit in this. I would thus like to thank the University of Applied Sciences Kiel for allowing me to take part in this exciting journey.

First of all, I would like to express my gratitude to Prof. Dr. Stephan Doerfel, who provided me with the opportunity to share my progress and activities and provided appropriate guidance and supervision.

I would also like to thank Christian Raddan for believing in this project and providing me with all the resources necessary to bring it to completion, whether that is connecting me with people, providing the context and background, or taking on the leadership role. Thanks to you, the project has gone smoothly with little to no hindrances.

A special thanks also goes out to Michael Hass and the ECC team, who have been pivotal to the quality of my work by providing the technical background on how the energy system works and by following up on every progress I have made. The level of expertise you bring to the table has inspired me to read further on the topic and ensure the project meets the quality expected. All those brainstorming hours, discussions, and reviews have led to the development of this project, and I am very thankful for your time.

Abbreviations

ACF	Autocorrelation Function
CHP	Combined Heat and Power
LP	Linear Programming
LSTM	Long Short-Term Memory
MAPE	Mean Absolute Percentage Error
MAX	Maximum
MBE	Mean Bias Error
MILP	Mixed-Integer Linear Programming
MIN	Minimum
PACF	Partial Autocorrelation Function
PCT	Percent
PV	Photovoltaic
RL	Reinforcement Learning
RMSE	Root Mean Square Error
SARIMA	Seasonal AutoRegressive Integrated Moving Average
SOC	State of Charge
STD	Standard Deviation
XGBoost	Extreme Gradient Boosting

Abstract

The project presents the outcomes of the forecasting and scheduling pipeline developed for the integrated energy system. The workflow combined electricity, heat, and PV forecasts with optimization-based (LP) and learning-based (RL) schedulers, compared against a baseline. The goal was to minimize total operational costs gas, electricity, and switching, while satisfying demand and respecting technical constraints. Forecasting, scheduling, feasibility checks, and control policy analysis were all conducted consistently over the test horizon, allowing systematic evaluation of performance improvements relative to the baseline.

For forecasting, XGBoost was selected as the best-performing model for electricity, heat, and PV, outperforming SARIMA and LSTM in test RMSE while capturing key temporal patterns. SARIMA provided interpretable seasonal structures but yielded non-normal residuals, and LSTM converged quickly but did not outperform XGBoost. In scheduling, the 24-hour LP achieved the lowest total cost before feasibility constraints were enforced, while enforcing feasibility slightly increased the total cost to satisfy all demand. RL produced low switching costs pre-feasibility but incurred higher total costs post-feasibility, partly due to differences in how high-level control variables map to physical demand satisfaction. Analysis of control variables showed that LP and RL implement dynamic, state-dependent policies: boilers are turned off when unnecessary, CHP is strategically modulated, the heat buffer is used flexibly, and battery discharge is minimal. Overall, cost-effective operation relies on adaptation to demand and forecasts, flexible use of storage, and selective operation of generation units, highlighting the

benefits of forecast-driven, optimization-informed control over static baseline approaches.

Contents

List of Figures

List of Tables

1. Introduction

Multi-energy systems in large buildings are complex to operate and often rely on fixed rules or operator experience. While these approaches are practical, they do not adapt well to changing energy prices, demand, and system conditions. This project aims to evaluate whether AI-assisted scheduling can reliably and practically reduce operating costs in such systems.

The project is carried out in collaboration with industry partners and focuses on a single pilot building site in Rendsburg Werkstätten. This is a workshop where people with disabilities work with the help of caring employees. The areas cover a metal workshop, a wood workshop (cabinetmaking), a print shop, and a canteen kitchen.

The site has multiple energy sources, including a compound Heat and power (CHP) system, a battery, an electric heating element, solar power, a thermal buffer, and a boiler. These energy sources are used to meet the heat and electricity demand of the building

All evaluation is performed in simulation to avoid risks to real-world operation. The outcome is intended to support decision-making on whether such approaches can be scaled to larger portfolios of buildings.

The objective of the study is to quantify the operating cost reduction achieved through AI-assisted scheduling compared to baseline operation in the pilot site. Specifically, the project investigated the operating cost reduction of the Reinforcement learner optimizer compared to the baseline and a linear programming optimizer.

Classical optimization methods for buildings include linear programming (LP) and mixed-integer linear programming (MILP), which are widely used and proven mathematical planning approaches [1]. Reinforcement learning (RL) is increasingly applied in energy system optimization because it can improve efficiency, reduce costs, and enhance sustainability. Unlike classical methods, RL adapts to complex, dynamic environments, making it suitable for energy management, grid control, and renewable integration [2]. Our thesis will compare these approaches to discern key distinctions in performance and implementation.

2. Literature Review

For the system design, the physical behavior of the various energy components has been simplified to enable modeling. The main simplification assumes linear input-output relationships under operating conditions, meaning that, for a given input to an energy component, the output is proportional and predictable. In multi-energy system modeling frameworks, complex conversion processes and storage behaviors are often approximated with piecewise-linear or constant-efficiency representations, so that the nonlinearities of real physical processes do not prevent the formulation of a linear or mixed-integer linear model. For example, the research [3] uses the Douglas-Peucker algorithm to linearize load curves. This research will use a constant-efficiency load curve that assumes constant efficiency across the operating range of the energy systems.

MILP is widely recognized as the state-of-the-art approach for optimizing polygeneration systems due to its ability to guarantee global optimality for linear problems, efficiently handle discrete decisions (e.g., unit on/off, technology selection), and utilize commercial solvers that scale to large problems. Typically, the model solves,

$$
\begin{aligned}
&\text{minimize} && f^T x && \text{objective function} \\
&\text{subject to} \\
&\{ \\
&Ac \cdot x \leq b && && \text{inequality constraints} \\
&A_{\{\text{eq}\}} c \cdot x = b_{\{\text{eq}\}} && && \text{equality constraints} \\
&l \leq x \leq u && && \text{variable bounds} \\
&y \in \{0, 1\} && && \text{binary variables (MILP only)} \\
&P_{\text{out}} = \eta \cdot P_{\in} && && \text{linear efficiency relation for components} \\
&\}
\end{aligned}
$$

MILP models provide tractable optimization by setting linear constraints, handling both continuous and integer variables, and are suited to multi-energy systems, CHP, storage, and distributed energy resources. However, they do not model nonlinear physical effects directly, and the computational complexity grows with system size [4].

Proximal Policy Optimization (PPO) is a reinforcement learning method for finding optimal control policies in complex systems. It models the system as a Markov Decision Process (MDP), where at each time step the agent observes a state, takes an action, and receives a reward [5]. The goal is to maximize the cumulative discounted reward:

$$R_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$

PPO uses a policy network (actor) to select actions and a value network (critic) to evaluate them. The advantage function estimates how much better an action is compared to the average. The actor network is updated by

increasing the probability of actions that lead to higher rewards. PPO stabilizes training using a clipped objective, which prevents excessively large updates. The critic network is updated to minimize the difference between predicted and actual rewards.

PPO can handle continuous, high-dimensional action spaces and stable policy updates thanks to clipping; however, it requires many trajectories and is sensitive to hyperparameters.

Multiple models were also investigated for forecasting. SARIMA models are classical statistical forecasting models that extend ARIMA to capture seasonal patterns in time series data. It is widely used in energy forecasting (e.g., load or price prediction), and is one of the benchmark models compared against machine learning approaches in the literature[6]. SARIMA models are usually written as:

$$\mathrm{SARIMA}(p,d,q)(P,D,Q)_s$$

Where :
The first triplet (p, d, q) is the non-seasonal orders:
p: number of autoregressive terms
d: order of differencing (trend removal)
q: number of moving average terms

The second triplet (P, D, Q) is the seasonal orders:
P: seasonal autoregressive order
D: seasonal differencing order
Q: seasonal moving average order
s: the seasonal period (e.g., 24 for hourly data with daily seasonality, 12 for monthly with yearly seasonality)
Mathematically, the non-seasonal component (ARIMA) can be described as:

$$X_t = \sum_{\{i=1\}}^{\{p\}} \varphi_i X_{\{t-i\}} + \sum_{\{j=1\}}^{\{q\}} \theta_j \varepsilon_{\{t-j\}} + \varepsilon_t$$

SARIMA models are well-understood, interpretable, and have clear statistical meaning; however, they assume linearity and stationarity of the data.

XGBoost (eXtreme Gradient Boosting) is an advanced gradient boosting framework for decision trees. It builds an ensemble of trees where each new tree is trained to correct the errors of the previous ones, optimizing a specified objective function using gradient descent. XGBoost offers high predictive accuracy, particularly for structured or tabular datasets, and is well-suited for capturing nonlinear relationships and complex feature interactions [7]. However, XGBoost requires careful hyperparameter tuning to achieve optimal performance, may still overfit if not properly regularized, is less interpretable than linear statistical models, and can be computationally more demanding compared to simpler approaches.

A Long Short-Term Memory (LSTM) network is a type of recurrent neural network (RNN) designed to model sequential data and maintain information over long sequences. Traditional RNNs struggle with long-range dependencies due to vanishing gradients during training; LSTMs address this by introducing memory cells and gating mechanisms that regulate the flow of information through time.

$$f_t = \sigma\big(W_f[h_{t-1}, x_t] + b_f\big) \quad \text{forget gate}$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{input gate}$$

$$c_t^{\sim} = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad \text{candidate cell state}$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot c_t^{\sim} \quad \text{cell state update}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{output gate}$$

$$h_t = o_t \cdot \tanh(c_t) \quad \text{hidden state output}$$

Where, x_t: input vector at time step, h_{t-1}: previous hidden state, σ: sigmoid activation producing values in [0,1] $\tanh$: hyperbolic tangent activation, W_* and b_*: learned weights and biases, and $\cdot$:element-wise multiplication [8]

LSTMs are capable of storing and retrieving information over long sequences, addressing long-term dependencies better than standard RNNs due to gated memory cells that preserve gradients during backpropagation; however, they are computationally intensive, require larger data, and are less interpretable than other model counterparts.

3. Methodology

For multi-energy system control, first, a deep understanding of the underlying structure and network is required. This will provide a baseline for further analysis. Thus, the first step was to develop a digital twin to enable modelling and simulation of the multi-energy system network. A digital twin, in our case, refers to the digital replica of the physical system that captures the main processes, assuming constant efficiency. The system description section details this process, including the energy systems, the data, and the baseline operation.

Once a key understanding of baseline operation was established, Optimization techniques were introduced into the system to observe any performance improvements. For this, MILP and PPO RL models have been experimented with to determine which scheduling method yields the lowest overall cost. A detailed description of the optimization methodologies is provided in the optimization section.

To make the optimization decision, the information horizon the optimizer receives should be future-looking, as current actions affect future decisions, and so an optimal decision has to be made over the entire horizon, not only at the current time. This calls for implementing a forecasting model to provide predictions that the optimizer can use to anticipate future events and make decisions that minimize cost over the entire horizon. Thus, multiple forecast models have been tuned and compared to identify the best-performing model. A deep-dive in the methodology and results has been presented in the forecast section.

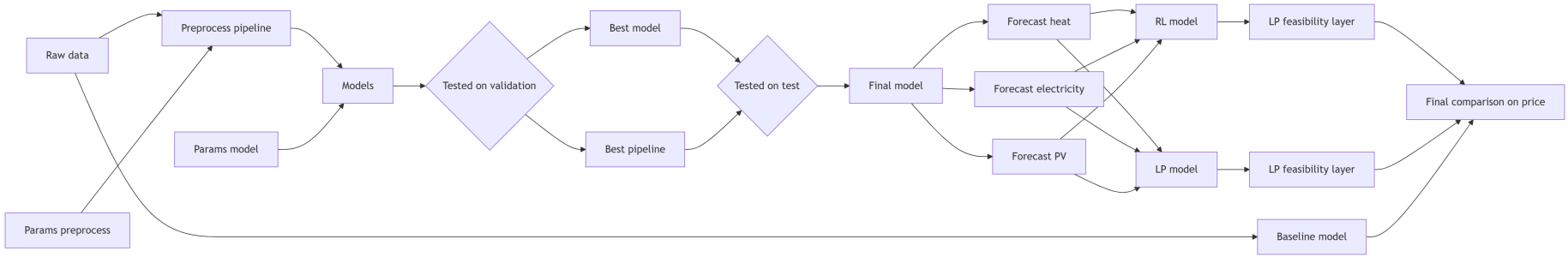

Figure 1: Overall Design Methodology.

4.System Description and Baseline Operation

The building has multiple energy systems that have different operation capacities and characteristics that will be described here. In addition, the data used, specifically, the energy demand, heat demand and solar generation will be processed and examined.Finally, the combined baseline operating principle is defined and simulated in order to assess the system's energy supply behavior under the given demand conditions.

4.1 The Energy System

The energy system has 6 main components. The CHP, the boiler, the electricity heating element, a thermal buffer, a battery and a solar panel. The boiler, electricity heating element and the thermal buffer provide heat to the building while the battery and solar panel provide electricity to the building. The CHP is a coupled system that can provide both heat and electricity to the building. If the system is unable to meet the total electricity demand, the deficit is supplied by the public power grid. Conversely, if the building generates electricity, the system has the option to feed the excess electricity to the public grid, thereby generating additional revenue. A Combined Heat and Power (CHP) unit generates electricity via a gas engine while recovering exhaust heat for building heating. A boiler and an electric heating element provide additional thermal energy when needed, with the boiler using fuel combustion and the heating element converting electricity directly into heat. A thermal storage (buffer) stores excess heat in an insulated tank, decoupling heat generation from demand. On the electrical side, a battery

stores electricity to balance supply and demand, while solar panels generate electricity from sunlight, supplying the building or feeding surplus into the grid. Each of these components has operational capacities and efficiencies that determine how they operate. A detailed operational capacity of each of these components can be described in the following table.

Component	Description	Value	Unit
CHP	Rated input gas	43.33	KW
	Rated output electricity	13	KW
	Rated output heat	26	KW
	total efficiency	90%	pct
	Operating Range	60-100%	pct of rated output
Thermal storage (Buffer)	Thermal capacity	100	KWh
	efficiency	95%	pct
Battery	Electricial capacity	60	KWh
	maximum power for charging/discharging	30	KWh
	efficiency	95%	pct
Photo-voltaic Panel (PV)	Rated Power at maximum radiation	110	KW
Gas Boiler	Rated thermal output power	150	KW
	Input Gas consumption at rated output	167	KW
	Operating Range	20-100%	pct of rated output

Component	Description	Value	Unit
Electric heating element	Rated input electricity consumption	30	KW
	Thermal energy output at rated input	29.4	KW

Table 1: Technical specifications of installed energy components.

These system components exhibit operational interdependencies, especially with respect to charging dynamics and interaction with the electricity heating element. The CHP and the solar panel provide the electricity supply for charging. The thermal buffer gets heat supply from the CHP and the boiler. The electricity heating element converts electric energy to heat energy and the electricity for the heating element is provided by the CHP, the solar panel and the battery. The following figure summarizes the interaction of these energy system to provide the appropriate energy supply.

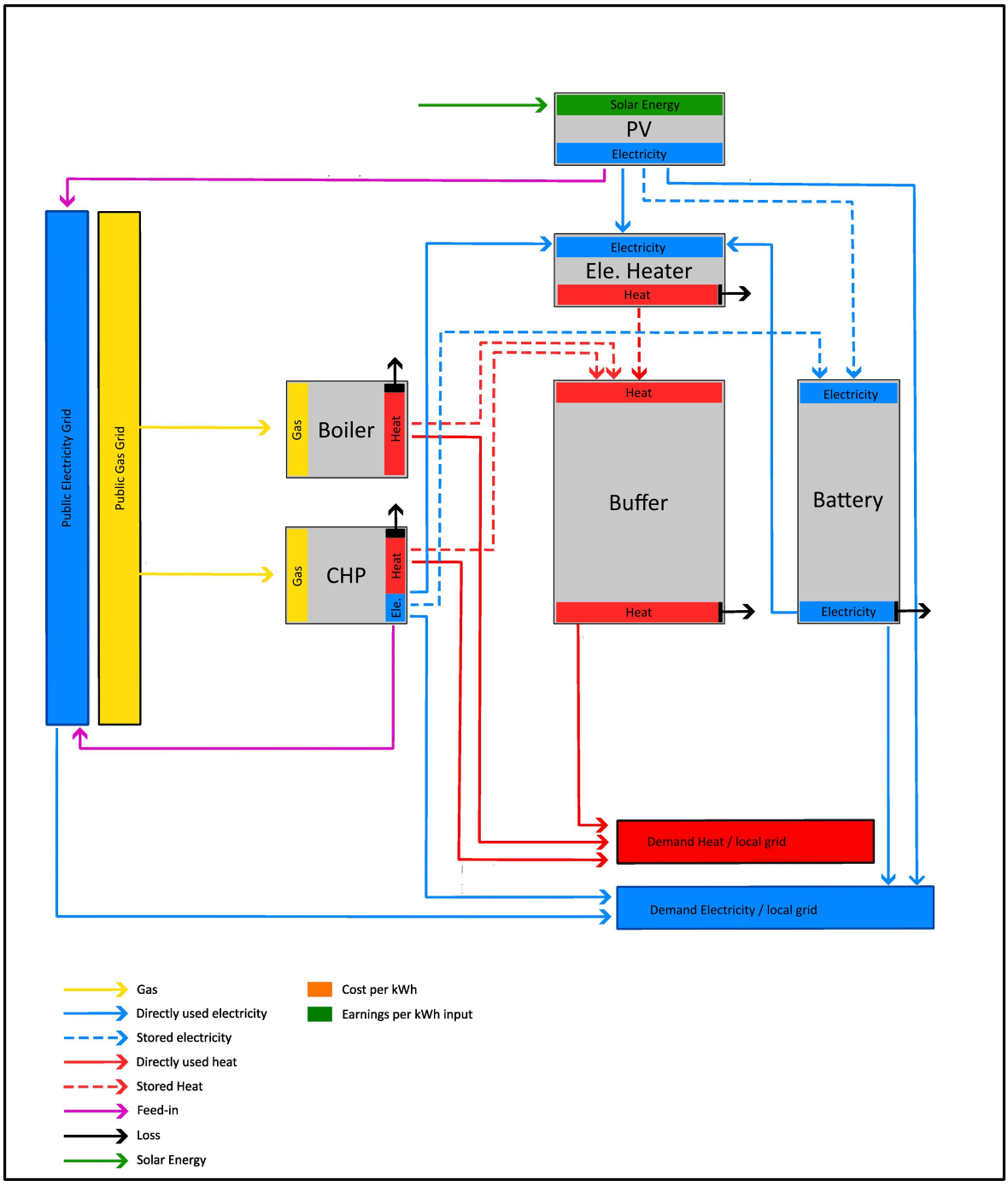

Figure 2: System layout of the case study building. (Provided by the consulting firm ECC)

4.2 The Data

Historical time series data of electricity demand, heat demand, and solar panel production has been used for generating forecasting models and optimization environments. The data has been collected from on-site sensors

and extracted from the company's proprietary data portal and can be extracted as either an `.xlsx` or a `.csv` file.

Preliminary Data Analysis

Before performing further analysis, the data has been wrangled to remove duplicates, check if index is sorted, ensure consistent frequency, check for null values, and check for their data types. Summary results can be found in the table below.

Description	Gas Electricity Consumption.xlsx	PV generation.csv
Shape	(35137, 2)	(35137, 2)
Number of Duplicated Timestamps	4	4
Number of Observed Frequencies	2	2
Number of null values	0	0

Table 2: preliminary data analysis results.

Our preliminary data analysis indicates that the dataset is relatively clean, requiring only minor adjustments before further analysis. After removing duplicate timestamps and standardizing the temporal frequency to one observation through reindexing and resampling (using the mean for aggregation), the dataset is prepared for subsequent analysis.

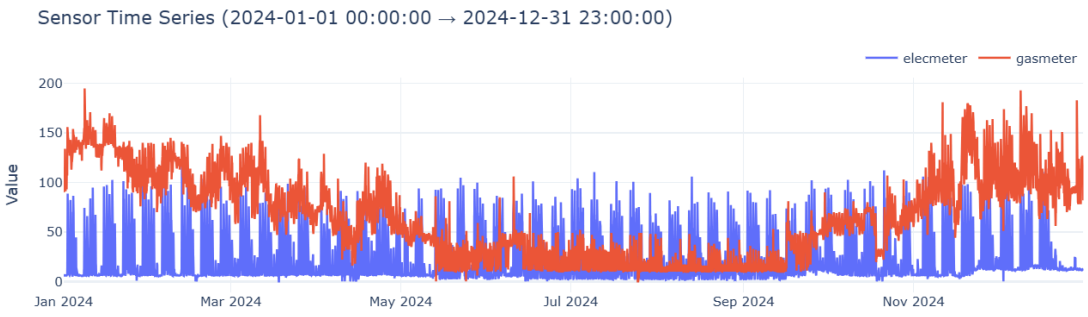

Figure 3: Hourly electricity and heat consumption [KW] in year 2024.

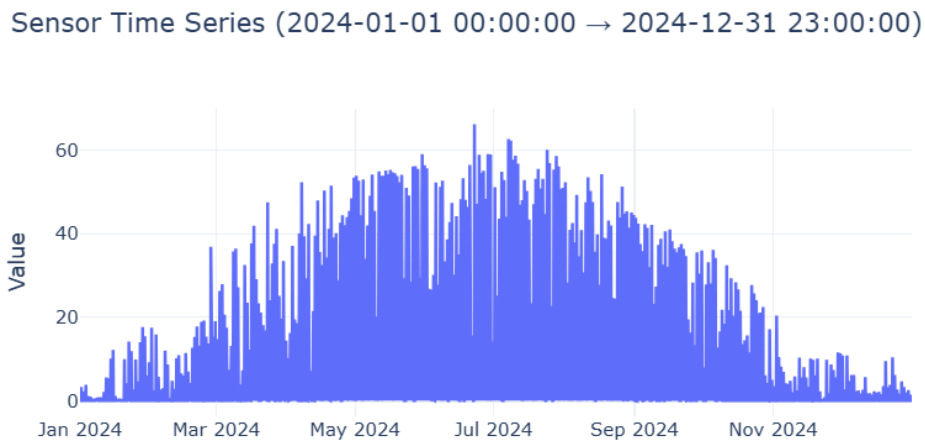

Figure 4: Hourly PV generation [KW] in year 2024.

Exploratory Data Analysis

To prepare for forecasting and subsequent analysis, it is essential to establish a fundamental understanding of the dataset. This enables verification of the assumptions underlying the applied models and facilitates the identification of potential challenges that may arise during the modeling process. Thus, a thorough exploratory data analysis with structural pattern detection, data quality identification, and distributional properties has been examined.

Description	electricity consumption	Gas Consumption	PV Generation
count	8784	8784	8784
mean	19.6	65.6	7.4
std	22.3	42.1	12.7
min	0.0	0.0	−0.0
25% Quantile	6.1	29.0	0
50% Quantile	10.4	60.0	0.1
75% Quantile	20.3	99.0	9.5
max	119.8	195	66.3

Table 3: Statistic for Gas Consumption, Electricity Consumption and PV Generation

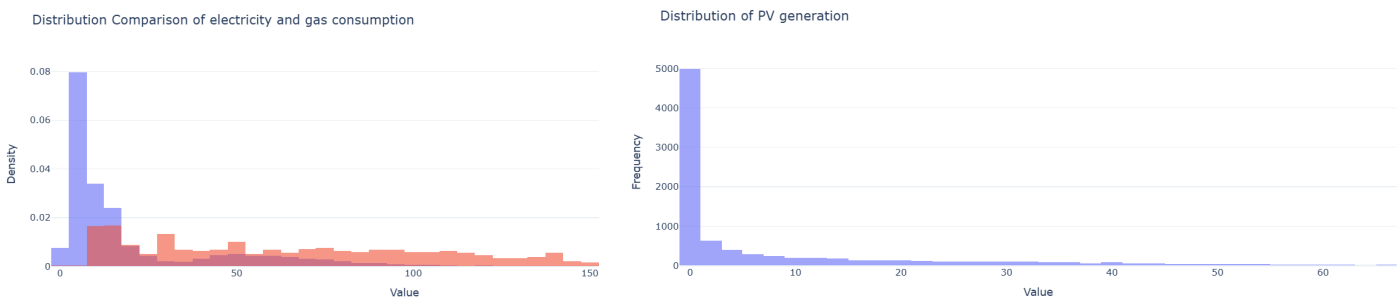

Figure 5: Distributions of Electricity and Gas Consumption and PV Generation.

The gas meter appears to be uniformly distributed, with the mean and the median having closer relationships; however, with some right tails observed, while the electricity meter is very right-skewed, with right end tails pushing the mean to the right compared to the median.

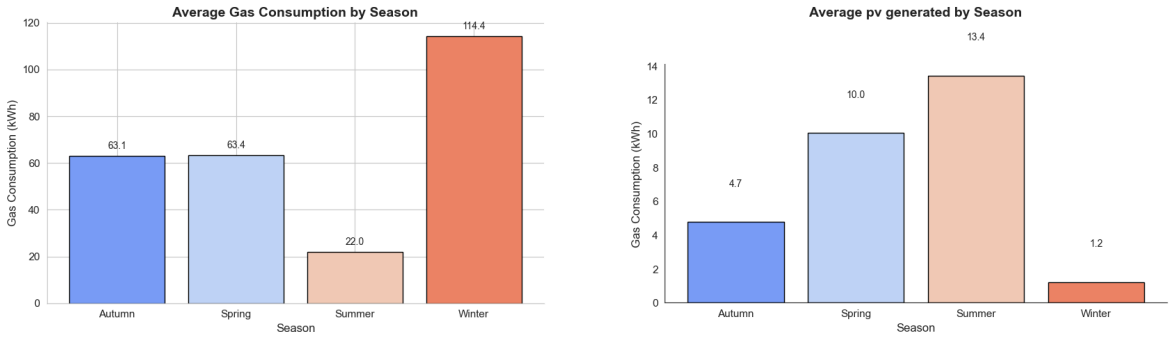

Figure 6: Average gas consumption and pv generated by season.

There are swings in max values for both electricity consumption and gas consumption that are significantly higher than the mean; however, about 75% of the data is somewhat closer to the mean, which implies there might be anomalies and outliers to detect. The variance of gas consumption is relatively higher than that of electricity consumption. From an aggregate point of view, the gas consumption and PV generated are consistent with typical building energy systems, where PV generation is relatively high in the summer and gas consumption relatively low in the winter.

4.3 Baseline Operation

The current operation model for the energy system is a rule-based methodology where energy systems are ranked by cost, and priority of energy supply is given to the one that has low cost.

PV is assumed to have no operational costs and thus given the first priority in supplying electricity demand. If PV cannot provide the requested demand, then CHP will provide the demand. If the combined CHP and PV generated electricity cannot provide the electricity demand, then the battery is allowed to discharge to provide extra supply. If there is still deficient demand, then the system will extract electricity from public power grid.

For heat demand, electricity heating element is given the first priority as it is assumed to leverage the electrical energy already available to be used for heating. The CHP will then be given priority. If there is still deficit in heat generated to meet demand, then the thermal buffer will be allowed to discharge heat. The boiler will be given the last priority should there still be unmet heat.

The gas price, electricity price, feed-in revenue and switching costs are also required to fully describe our system. These can be analyzed in a dynamic price environment or a static price environment. Dynamic prices imply prices will continuously change in a way that might change decisions of system scheduling while static prices imply the contrary. Static prices will be used thought this project as most prices is not assumed to drastically change and most of them are already static in nature except for electricity prices where it is already known that there are already spot markets available for dynamic

pricing. The following table shows the static prices assumed for the year 2024.

Given the historical data, the rules, the prices and the different energy components specifications (Table 1) We can simulate how these rules affect the operational cost of the energy system. The following Algorithm shows how the operation cost under these rules is implemented

```
Algorithm: Baseline Operation
-------------
Inputs (per hour t):
  - heat_demand[t], elec_demand[t], pv_generated[t]
  - component models: CHP, boiler, electric heater (EE), battery, buffer, PV
  - current SOCs: battery.soc, buffer.soc
  - target SOCs: battery_target, buffer_target
  - prices: gas_price, grid_price, feed_in_price
  - flags: chp_charge_only

1. Allocate PV directly to electrical demand
   - Use PV up to elec_demand.
   - Compute remaining elec demand and PV surplus.

2. Plan electricity supply to direct load
   - Tentatively cover remaining elec demand with:
     a) CHP (up to its rated elec output)
     b) Battery discharge (up to SOC and power limits)
     c) Grid for any residual.

3. Plan electric heater (EE) heat and its electricity
   - Decide desired EE heat (bounded by EE capacity and heat demand).
   - Compute required EE electricity.
   - Allocate that electricity in order:
```

```
     a) remaining PV surplus
     b) additional CHP output
     c) battery discharge
   - Derive actual EE heat from supplied EE electricity.

4. Plan battery charging (to reach battery_target)
   - Compute how much additional energy you want in the battery.
   - Use remaining PV surplus and, if needed, extra CHP electricity
     to charge the battery (respecting power and SOC limits).

5. Plan total CHP gas input with min-load logic
   - Scenario A (electric-driven): sum of tentative CHP inputs from
     direct load, EE, and battery charging.
   - Scenario B (heat-driven): CHP run on the heat side to cover
     remaining heat demand (subject to CHP heat rating).
   - Enforce minimum load: if a scenario uses < 60% rated input, treat
     that scenario as “off”.
   - Choose the larger of the two scenarios (A vs B) as the actual
     CHP gas input for this hour.
   - Redistribute that chosen CHP input back into:
     - direct elec supply
     - EE supply
     - battery charging
     in proportion to their tentative electric shares.

6. Recompute electricity balances with the chosen CHP input max(scenario
A,scenario B)
   - Use CHP, PV, and battery to meet direct elec and EE electricity.
   - Anything left is drawn from the grid.
   - Any excess CHP/PV electricity beyond demand and charging is fed into
the grid.
```

```
7. Meet heat demand using EE heat, CHP heat, buffer, and boiler
   - Start from heat demand.
   - Subtract EE heat and the part of CHP heat assigned to demand.
   - Use buffer discharge to cover part of the residual heat demand.
   - Any remaining heat demand is covered by the boiler (up to its rating).

8. Decide heat storage in the buffer
   - Compute desired additional buffer SOC to reach buffer_target.
   - Use CHP excess heat to charge the buffer.
   - If allowed (not `chp_charge_only`), also use boiler headroom
     to charge the buffer, respecting boiler and buffer limits.
   - Enforce boiler minimum load: if total boiler input (demand + storage)
     falls below 20% of rated input, turn boiler off and do not store.

9. Update SOCs
   - Update battery SOC from all charge/discharge operations.
   - Update buffer SOC from all discharge/charge operations.

10. Economic evaluation
   - Compute gas consumed by CHP and boiler and multiply by gas_price.
   - Compute grid electricity purchases times grid_price.
   - Compute electricity fed into the grid times feed_in_price (negative
cost).
   - Aggregate to total gas and electricity costs for this hour.

11. Return
   - All component outputs (CHP, boiler, EE, PV, battery, buffer),
   - Final SOCs,
   - Total costs and any key diagnostic quantities (e.g. unmet heat, wasted
heat).
```

As we can observe from the algorithm, that there are two things we assume. The first is the initial state of charge (SOC) of the storing elements, secondly,

we are forced to charge our batteries to a certain desired SOC, as there are no explicit rules on when to charge or discharge, or store elements. So our rules currently have no options to handle this and is left as an assumption. Thus, we will test these assumptions by using sensitivity analysis, where we vary these assumptions to see the resulting effect on cost.

Description	Value	Unit	Comments
Gas Price	0.11	€/KWH	
Electricity Price	0.3	€/KWH	
Feed-in value	0.065	€/KWH	The revenue generated if electricity is fed to the grid
switching cost	10	€	The unit cost of maintenance as a result of switching our CHP and boilers from an off state to an on state

Table 4: static prices to calculate the total operational cost

Effect of Initializing SOC

Initalization of SOC has little to no effect on yearly trajectories as only only €19 of range is detected between the maximum and minimum total yearly cost. The experiment was conducted by sampling 5 different SOC values for the buffer ([0,10,30,40,60]) and the battery ([0,20,50,70,100]) and calculating the statistics of the resulting changes.

Effect of charging battery/buffer to a desired SOC

The effect of charging storage units at constant value does indicate interesting compromises between gas/electricity costs and switching costs. The graph Figure 7 shows as we increase the percentage of SOC, we are reducing the switching costs. This is because, as the storage units are stored

in the current timestep, the need for other units to provide all the power in the future will reduce. When the storage units have no charge left in them, then other units will have to provide the extra power. So we can see the direct benefit of storage units in this context. However, the overall cost monotonically increases as we are always charging the battery to a certain value, which is a cost-generating action. The current baseline model does not have enough rules to balance between when to charge the storage units and when not to charge the batteries.

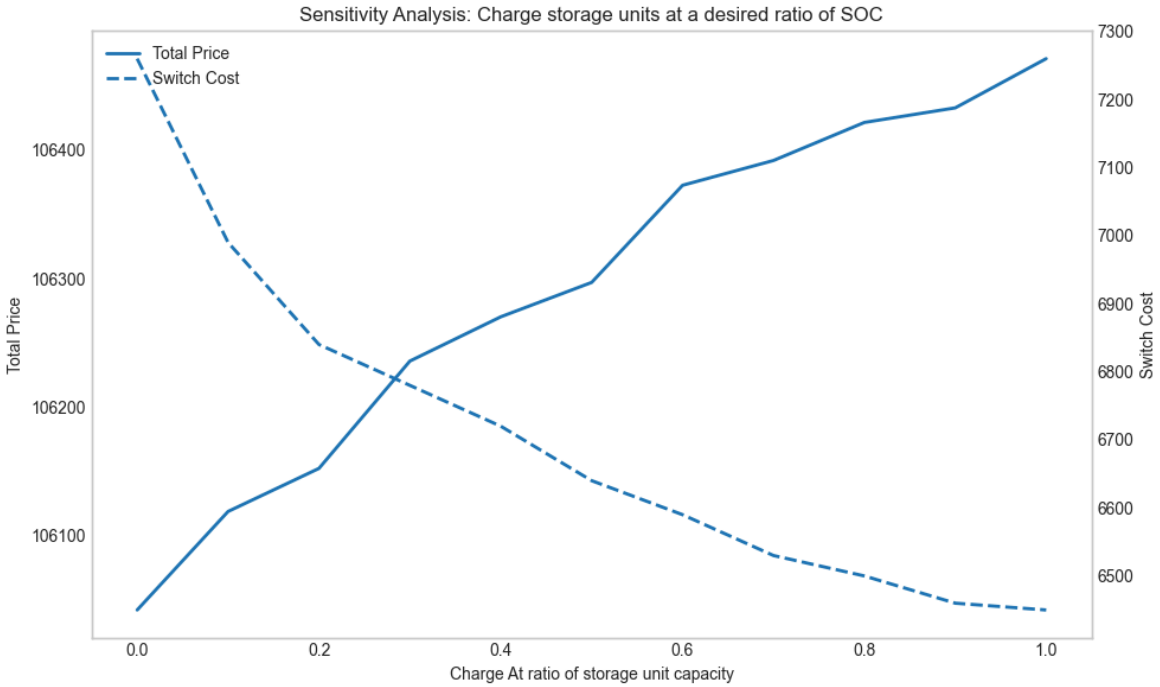

Figure 7: Sensitivity of price when storage units are constantly charged at a constant percentage of rated capacity.

5. Forecasting

This section presents the forecasting experiments conducted to predict photovoltaic (PV) generation, electricity demand, and heat consumption. The resulting forecasts serve as inputs to the optimization models developed in the subsequent sections. Accurate forecasting is a critical component of the overall framework, as the optimization models operate under a look-ahead assumption and base their decisions on anticipated future system states. Consequently, the quality of the forecasts directly influences the performance and reliability of the optimization outcomes.

The objective of this section is to compare and contrast different forecasting models that range from simple statistical baselines to advanced deep learning architectures, enabling an analysis of both linear and nonlinear temporal dependencies. Specifically, a persistence model serves as a benchmark, while SARIMA is used to capture seasonal patterns, and XGBoost and Long Short-Term Memory (LSTM) networks are evaluated for their ability to capture complex temporal patterns and exogenous influences. All models are trained and tested under a unified experimental framework to ensure comparability of results.

In the forecasting pipeline, time series of electricity demand, heat demand, and PV generation are provided as input. These series are indexed by timestamps and are first wrangled using a flexible data wrangling pipeline that summarizes key information about the data, handles missing values and data types, and provides frequency information. Based on the information provided, the data is preprocessed to remove duplicates, impute null values,

and correct frequency inconsistencies so that the data can have consistent time steps.

The data is then passed through a configurable preprocessing pipeline. Within this pipeline, calendar-based features (hour of day, day of week, month, etc.) are generated by a CalendarFeatures module, holidays are added via a GermanHoliday module (which uses an external package of `python import holidays`), external covariates are incorporated by a WeatherFeatures module (that uses an open API from open-meteo), lagged values are created by a LagFeatures module, and transformations such as logarithmic scaling and standardization are applied. Reshaping to sequences is optionally performed for sequence models, and a time-series splitter from a TimeSeriesSplitter class that uses a ratio of train, validation, and test splits or a splitter function that can be configured from a `.csv` file that stores the exact start and end time for train, validation, and test.

The overall orchestration of the forecasting implementation is found `forecast_helper.py`. In this module, pipelines are defined as combinations of a time-series splitter, a list of preprocessing steps, a model constructor, and a metric function. For each pipeline, the data are split, the preprocessing steps are fit on the training set and applied to validation/test sets, and the selected model is instantiated with given hyperparameters. The model is then trained on the training portion and evaluated on the validation or test portion using metrics such as root mean squared error (RMSE). This logic is encapsulated in helper functions like evaluate_model and run_pipelines_hyperparam, which are used to compare and select forecasting configurations.

Several forecasting model families are used, each backed by a specific library. SARIMA and SARIMAX models are implemented using the statsmodels library (e.g. statsmodels.tsa.statespace.SARIMAX). These models are classical statistical time-series models that assume a linear autoregressive–moving-average structure with seasonal components and, in the case of SARIMAX, exogenous regressors. Once configured (orders, seasonal orders, exogenous variables), they are fit by maximum likelihood and produce both in-sample and out-of-sample forecasts with probabilistic state-space machinery.

A gradient-boosted tree model is provided via XGBoost, accessed through the xgboost Python package (`import xgboost`). In this setting, tabular feature matrices—built from the preprocessor (calendar, lags, weather, etc.) are given to XGBoost, which then learns non-linear relationships between features and the target by sequentially fitting regression trees that correct the residuals of previous trees.

A recurrent neural network model (LSTM) is implemented using PyTorch (torch), where an LSTM_Model processes sequence-shaped inputs produced by the preprocessing pipeline (ReshapeToSequence). In this architecture, sequences of past observations and covariates are fed into one or more LSTM layers that maintain hidden states over time, allowing temporal dependencies to be modeled explicitly. The network parameters are learned via stochastic gradient descent (or related optimizers) with backpropagation through time, and the trained LSTM is then used to generate multi-step forecasts.

In addition, a persistence model is provided as a very simple baseline, which is typically implemented directly in Python or via a small custom class (e.g. PersistenceModel in the forecast_models module). This model often predicts the next value as the last observed value (or a simple shift/average), and is used to benchmark whether more sophisticated methods truly add value. All these models persistence, SARIMA/SARIMAX, XGBoost, and LSTM are evaluated within the same pipeline framework, and the best-performing combination of preprocessing and model is selected based on validation performance, then re-evaluated on the test set and finally used to generate the heat, electricity, and PV forecasts that are passed to the downstream optimization and RL components.

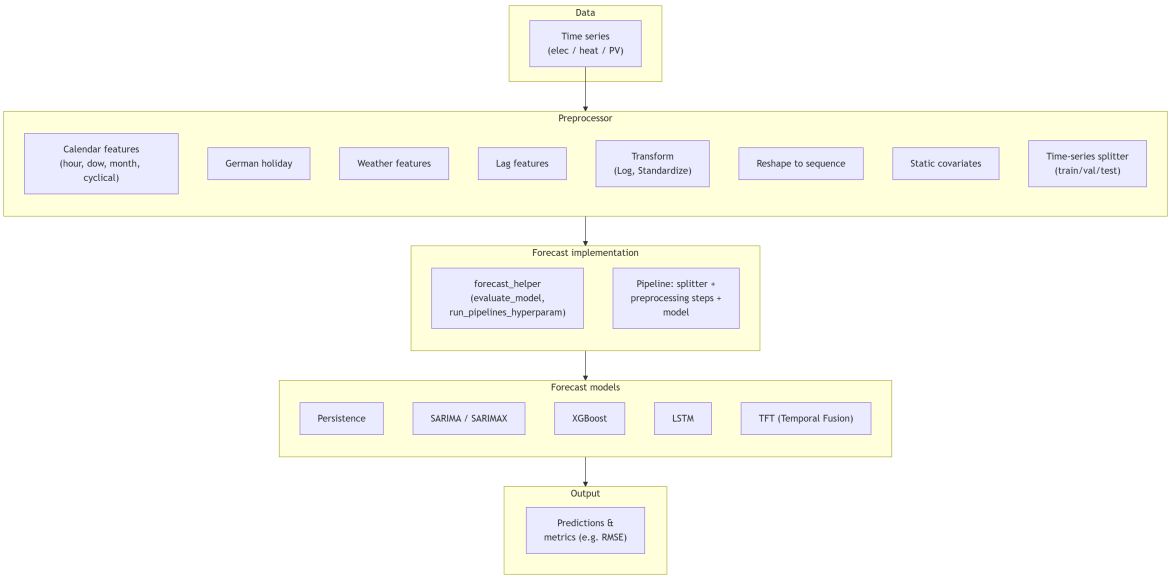

Figure 8: Forecast module Interactions.

5.1 Electricity Demand Forecast

The electricity demand data showed strong daily and weekly seasonality, showing higher peak values on weekdays and lower values on weekends. Some studies [9] show a strong relationship between electricity demand and

heat. However, our dataset reveals only a weak linear relationship, with a Pearson correlation coefficient of 0.13.

Given these parameters, the ARIMA model was configured to capture dominant daily seasonal patterns, and a SARIMAX specification was implemented to allow the inclusion of exogenous variables where appropriate.

For our XGBoost and LSTM models, more features have been added other than lag features. These features include the calendar and the weather. Lags have been selected based on strong values in the ACF and PACF.

For each modeling approach, a predefined hyperparameter search space was evaluated using grid search on a validation set. The best-performing configurations were subsequently assessed on a separate test set to ensure a robust model comparison.

Root mean square error (RMSE) is chosen as our primary statistic for model selection and evaluation because it is more sensitive to larger errors. Mean bias error (MBE) is chosen as our second statistic. This parameter tells whether our forecaster generally overestimates or underestimates. This is a very good indicator of our problem because it shows how our forecastor interacts with our optimized schedulers. If the forecaster consistently underestimates, our scheduler will also underestimate and will not fully meet the building’s demand. If the forecast overestimates, the scheduler will exceed actual demand.

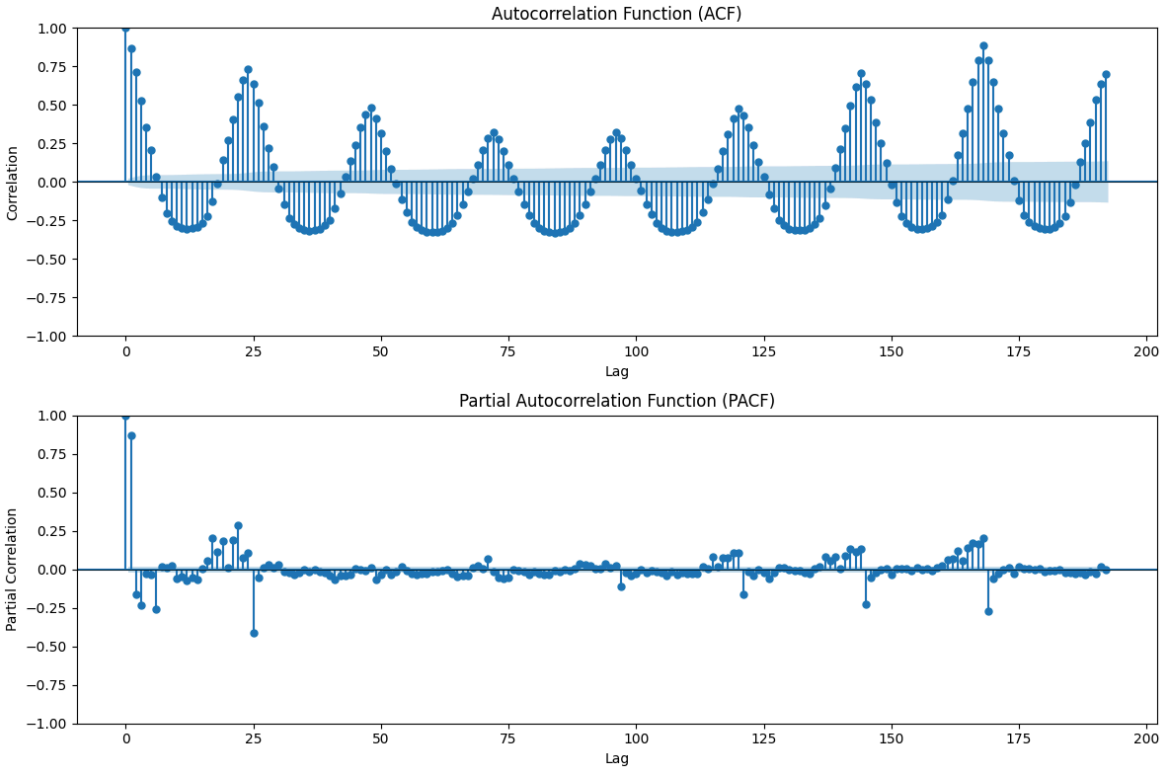

Figure 9: PACF and ACF graphs for electricity demand

model	rmse	mbe	elapsed_sec	train_rmse	train_mbe
XGBoost	5.88	-1.39	0.04	4.69	-0.00
LSTM	9.83	-2.11	4.05	6.23	0.13
SARIMA	18.40	-1.90	7.64	11.06	-0.07
Persistence	25.03	-11.87	0.00	11.45	0.00

Table 5: Summary results of electricity Forecast

Table 5 indicates that the XGBoost algorithm outperforms in all metrics. XGBoost has the highest root-mean-square error and the highest MBE, with faster compute times.

5.2 Heat Demand Forecast

Heat demand is uniquely driven by temperature dependence and seasonal structure, with demand largely concentrated in colder months and often near zero during summer. Unlike electricity, it typically lacks strong daily behavioral peaks and instead follows smoother, temperature-driven patterns.

The relationship between temperature and heat demand was found to be highly linear (correlation of −0.83), indicating that heat demand is more physically constrained and thermodynamically driven than electricity demand. As such, temperature parameters were used in all model pipelines.

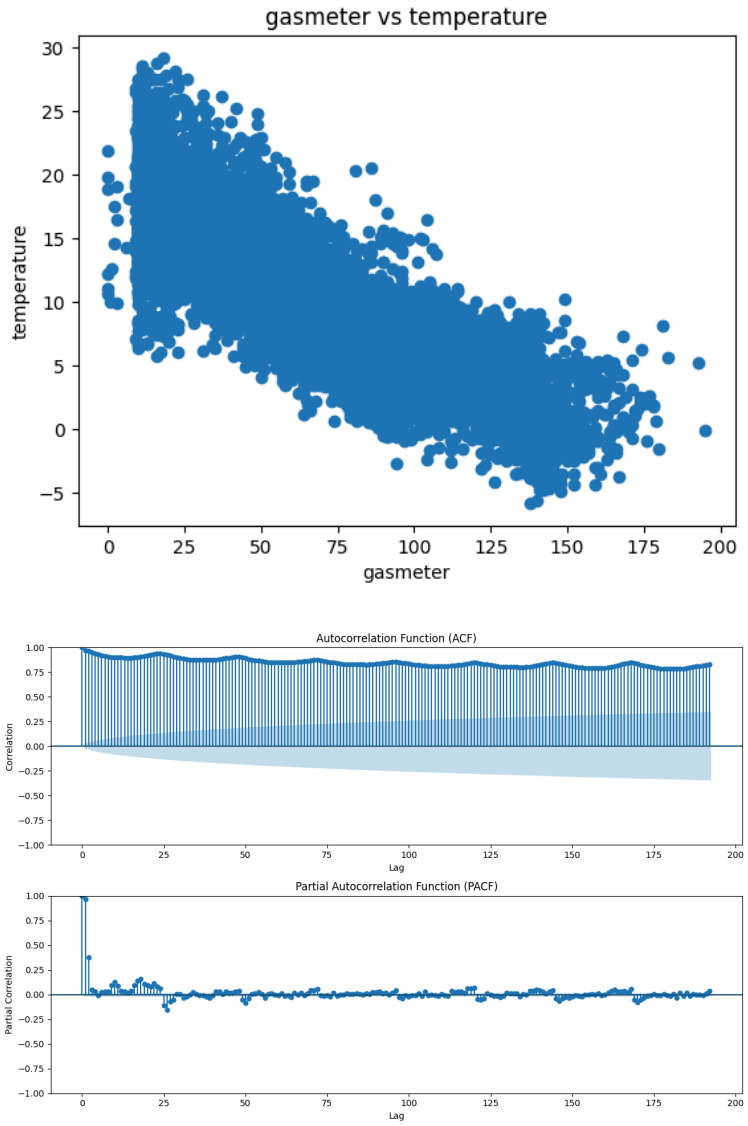

Figure 10: Correlation,PACF and ACF graphs for heat demand

model	rmse	mbe	elapsed_sec	train_rmse	train_mbe
XGBoost	11.20	-0.30	0.03	9.09	0.00
LSTM	15.75	-1.03	9.75	7.94	-0.17
SARIMA	74.46	-67.71	6.18	15.65	0.36
Persistence	78.18	-71.17	0.00	10.19	0.02

Table 6: Summary results of Heat Forecast

5.3 PV Generation Forecast

PV generation is fundamentally supply-driven and governed by solar irradiance rather than human behavior or temperature. For our data, there was a 0.9 correlation between solar irradiance and a 0.5 correlation between temperature. It exhibits strict diurnal cycles, with zero production at night; highly predictable annual seasonality driven by solar geometry; and strong short-term volatility driven by cloud cover and atmospheric conditions. Unlike electricity or heat demand, PV output is bounded by installed capacity and is subject to a clear physical upper limit determined by panel orientation and efficiency. Rapid ramping events caused by passing clouds create high-frequency fluctuations that are particularly challenging to forecast. As such, more weather parameters, such as cloud cover and solar irradiance has been added to our features.

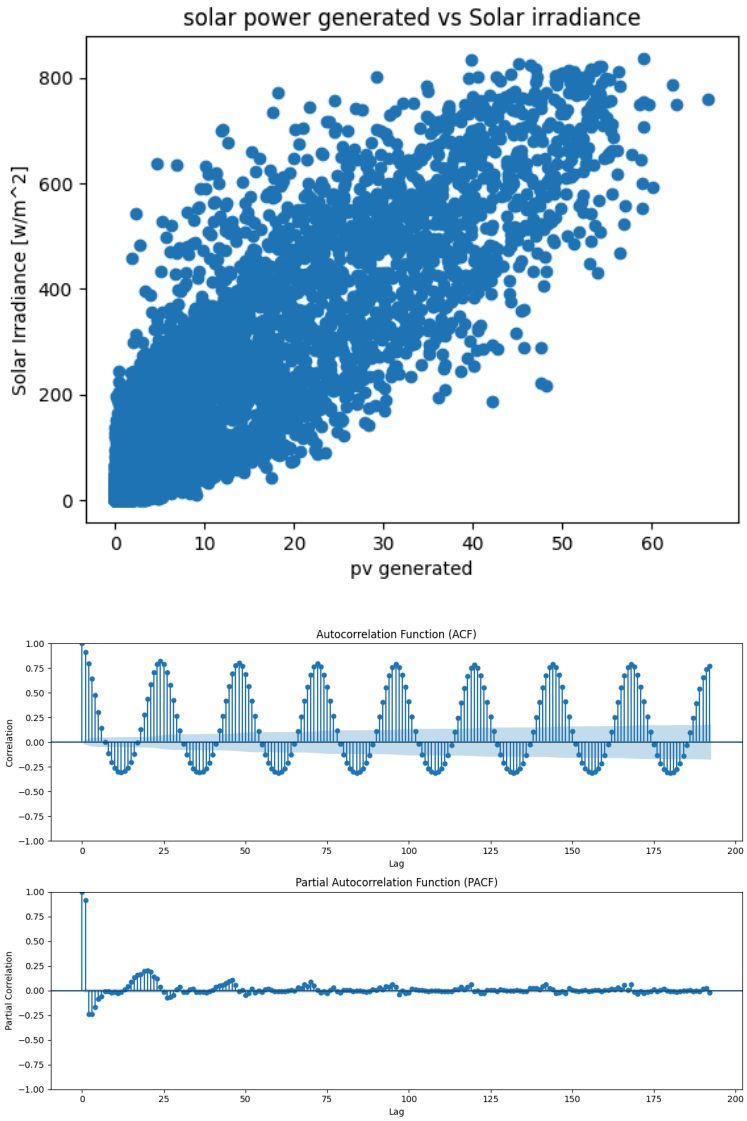

Figure 11: Correlation,PACF and ACF graphs for PV

model	rmse	mbe	elapsed_sec	train_rmse	train_mbe
XGBoost	1.66	-0.16	0.02	3.10	-0.00
LSTM	2.44	0.61	7.38	4.02	0.20
SARIMA	19.02	13.42	5.21	4.34	-0.25
Persistence	23.78	16.27	0.00	5.50	0.00

Table 7: Summary results of PV Forecast

6. Optimization

This section presents the optimization methodology developed to determine the optimal operational scheduling of the energy system components. Building upon the forecasting results described in the previous section, the optimization framework translates predicted electricity demand, heat demand, and PV generation into operational decisions subject to technical and system constraints.

One of the main objectives of the thesis is to transfer rule-based scheduling methodologies to more dynamic scheduling approaches, including linear programming and Reinforcement learning.

To determine the most suitable scheduling approach, the candidate optimization strategies are systematically compared. In addition, a baseline model reflecting current operational guidelines is employed to quantify the relative performance improvements achieved by the proposed optimization methods.

The evaluation is conducted over the final three months of the year, corresponding to the test horizon used in the forecasting stage. This ensures methodological consistency between forecast validation and optimization assessment. For each model, appropriate pipelines were developed with detailed logging of each component for later analysis, debugging, and verification. For the LP and RL optimizer, a post-hoc LP feasibility layer is added that adjusts the optimizer’s plans to ensure heat and gas demand are met, including other physical constraints, and is subsequently investigated using a constraint-checking function.

In the optimization pipeline, time series of electricity demand, heat demand, and PV generation are combined with a parameterized description of the energy system and are transformed into operating schedules and economic evaluations. The physical system is represented by component classes (CHP, boiler, electric heating element, PV, battery, buffer, digital twin) that are defined in the energy_components module. These classes encapsulate technical parameters such as rated power, efficiency, and storage capacity, and are stored in a components dictionary that is passed to the optimizers and feasibility layers.

```
Algorithm: pseudocode for 24 hour LP optimizer
Inputs:

  date_time_index – start timestamp of 24h horizon
  frequency – time step in minutes (e.g. 60)
  components – boiler, chp, ee, pv, battery, buffer (with capacities,
efficiencies, min-load, etc.)
  elec_demand, gas_demand, pv_generated – full time series
  gas_price, grid_price, feed_in_price, switch_cost – economic parameters
Output:

  24-step schedule (flows, SOC, on/off, prices, switch flags, status)
indexed by timestamps
1. Extract data window and precompute parameters

  1.1 Set horizon length W = 24, Δt = frequency / 60.
  1.2 Find window start index start_pos from
elec_demand.index.get_loc(date_time_index).
  1.3 Slice 24-step windows:
      elec_win ← elec_demand[start_pos : start_pos+W]
      heat_win ← gas_demand[start_pos : start_pos+W]
```

```
      pv_win ← pv_generated[start_pos : start_pos+W]
  1.4 Clip to nonnegative: elec_win = max(elec_win, 0), heat_win =
max(heat_win, 0), pv_win = max(pv_win, 0).
  1.5 Read component parameters, initial SOC:
      soc_batt[0] ← clipped(battery.soc_elec)
      soc_buff[0] ← clipped(buffer.soc_heat)
  1.6 Precompute CHP heat/electricity per fuel unit.

2. Define decision variables for each hour i = 0..W-1

  Heat flows: chp_hd[i], boiler_hd[i], buffer_to_discharge_hd[i], ee_hd[i]
  Electric flows: pv_ed[i], battery_to_discharge_ed[i], chp_ed[i],
grid_out[i]
  Couplings and charges:
      chp_batt[i], pv_batt[i], chp_buff[i], boiler_buff[i],
      chp_ee[i], pv_ee[i], batt_to_discharge_ee[i],
      batt_to_charge[i], buff_to_charge[i], ee_ed[i],
      batt_to_discharge_total[i]
  CHP internals: chp_in[i], chp_heat_total[i], chp_elec_total[i]
  SOC trajectories: soc_batt[i+1], soc_buff[i+1]
  Feed-in: pv_feed_in[i], chp_feed_in[i]
  Binary on/off and switching: chp_on[i], boiler_on[i], chp_switch[i],
boiler_switch[i]
  Unmet slacks: unmet_heat[i], unmet_elec[i] (for 24h LP they are present
and heavily penalized)
  (All are ≥ 0 except binaries.)

3. Add constraints for each hour i

  3.1 Heat and electricity balances:
      chp_hd + boiler_hd + buffer_to_discharge_hd + ee_hd + unmet_heat =
heat_win[i]
```

pv_ed + battery_to_discharge_ed + chp_ed + grid_out + unmet_elec = elec_win[i]

3.2 Linking:

chp_batt + pv_batt = batt_to_charge

chp_buff + boiler_buff = buff_to_charge

pv_ee + chp_ee + batt_to_discharge_ee = ee_ed

3.3 PV limit:

pv_ed + pv_ee + pv_batt ≤ pv_avail_i = min(pv_win[i], pv.capacity)

3.4 Boiler (20–100% or off):

boiler_out_total = boiler_hd + boiler_buff

boiler_out_total ≤ boiler_on · boiler.rated_output_heat

boiler_out_total ≥ 0.2 · boiler.rated_output_heat · boiler_on

3.5 EE:

ee_ed ≤ ee.rated_input_elec

ee_hd = ee_ed · (ee.rated_output_heat / ee.rated_input_elec)

3.6 Battery:

batt_to_discharge_total = batt_to_discharge_ee + battery_to_discharge_ed

batt_to_discharge_total ≤ soc_batt[i] · η_discharge

batt_to_discharge_total ≤ max_discharge · Δt · η_discharge

batt_to_charge ≤ (capacity − soc_batt[i]) · η_charge

batt_to_charge ≤ max_charge · Δt · η_charge

3.7 Buffer:

buffer_to_discharge_hd ≤ soc_buff[i] · η_discharge

buff_to_charge ≤ (capacity − soc_buff[i]) · η_charge

3.8 CHP with 60% min-input:

chp_in ≤ chp_on · rated_input_heat

chp_in ≥ 0.6 · rated_input_heat · chp_on

chp_heat_total = chp_in · heat_per_fuel

chp_elec_total = chp_in · elec_per_fuel

chp_heat_total ≥ chp_hd + chp_buff

chp_elec_total ≥ chp_ed + chp_ee + chp_batt

chp_heat_total ≤ rated_output_heat, chp_elec_total ≤ rated_output_elec

3.9 SOC dynamics:

soc_batt[i+1] = soc_batt[i] + batt_to_charge · η_charge − batt_to_discharge_total / η_discharge

soc_buff[i+1] = soc_buff[i] + buff_to_charge · η_charge − buffer_to_discharge_hd / η_discharge

0 ≤ soc_batt[i+1] ≤ capacity, 0 ≤ soc_buff[i+1] ≤ capacity

3.10 Feed-in:

pv_feed_in ≤ pv_avail_i − (pv_ed + pv_ee + pv_batt)

chp_feed_in ≤ chp_elec_total − (chp_ed + chp_ee + chp_batt)

3.11 Switching logic:

For i = 0:

chp_switch[0] ≥ chp_on[0]

boiler_switch[0] ≥ boiler_on[0]

For i > 0:

chp_switch[i] ≥ chp_on[i] − chp_on[i-1]

boiler_switch[i] ≥ boiler_on[i] − boiler_on[i-1]

4. Objective (minimize cost with penalties)

4.1 Fuel and electricity cost:

chp_fuel_cost = sum(chp_in) · Δt · gas_price

```
      boiler_fuel_input = sum(boiler_out_total) · (rated_input_heat /
rated_output_heat)
      boiler_cost = boiler_fuel_input · Δt · gas_price
      grid_cost = sum(grid_out) · Δt · grid_price

  4.2 Feed-in revenue:
      feed_in_revenue = feed_in_price · Δt · sum(pv_feed_in + chp_feed_in)

  4.3 Switching and unmet penalties:
      total_switch_cost = switch_cost · (sum(chp_switch) +
sum(boiler_switch))
      unmet_penalty = big_penalty · (sum(unmet_heat) + sum(unmet_elec))

  4.4 Objective:
      minimize chp_fuel_cost + boiler_cost + grid_cost − feed_in_revenue +
unmet_penalty + total_switch_cost

5. Solve and build output

  5.1 Solve the MIP with GUROBI via CVXPY; capture status and solver log.
  5.2 Extract all variables into arrays (boiler, Grid, chp_out_*, pv_out_*,
buffer_out, batt_out_*, soc_elec, soc_heat, chp_on, boiler_on, chp_switch,
boiler_switch, etc.).
  5.3 Compute hourly prices (gas, electricity, total, feed-in revenue).
  5.4 Build a 24-row DataFrame indexed by the 24 timestamps, containing
flows, SOC, prices, on/off and switch flags, status, solver_used,
solver_verbose, and end-of-horizon SOC (soc_elec_end, soc_heat_end).
```

Two main scheduling approaches are used: a linear-programming (LP) scheduler and a reinforcement-learning (RL) scheduler, alongside a simple baseline. The LP scheduler is implemented in the Schedulers and Optimization modules. In particular, functions such as

`optimize_24h_with_switching_cost` in `lookahead_lp_optimizer_switch.py` use the `cvxpy` optimization library to formulate and solve a 24-hour linear program. In this formulation, decision variables are introduced for electricity and heat flows (e.g., CHP outputs, boiler outputs, buffer charges/discharges, battery charges/discharges, grid imports/exports), and linear constraints enforce power balance, efficiency relations, storage dynamics, and minimum-load or on/off behavior for units like the CHP and boiler. An objective function is constructed to minimize total operating cost, which includes gas costs, grid electricity costs, feed-in revenues, and, when enabled, switching costs. The `cvxpy` package is used to express this optimization problem symbolically and to call numerical solvers that compute the optimal schedule over the 24-hour horizon. A runner (`run_sliding_24h_forecast in look_ahead_lp_runner.py`) is then used to apply this 24-hour optimization sequentially across the full time series, producing a concatenated schedule.

The RL-based scheduler is implemented via a custom Gymnasium environment, defined in `Schedulers/look_ahead_env_for_RL.py`. The gymnasium (OpenAI Gym successor) library is used to provide a standard reinforcement-learning interface with reset and step methods, a continuous action space, and an observation space that contains demand forecasts, PV availability, and state-of-charge (SOC) information. At each time step, 5 actions are interpreted as continuous control decisions (e.g. CHP loading, boiler loading,boiler_to_buffer, buffer discharge, and battery discharge), and the environment simulates resulting electricity and heat balances, updates the storage states, and computes a reward. The reward is designed to penalize fuel and grid costs, unmet heat demand, wasted heat, and equipment switching, encouraging a learned policy to find cost-effective and physically

reasonable schedules. External RL algorithms (e.g., from Stable-Baselines3) were attached to this environment to learn policies over episodes.

```
Algorithm pseudocode for step in look_ahead_env_for_RL
-------------
Inputs:
  action – 5-dim continuous vector from policy (raw in ([-1,1]))
  Internal state: episode_t, start_hour, soc_batt, soc_buff, prev_chp_on,
prev_boiler_on
  Time series: elec_demand, heat_demand, pv_gen
  Component parameters: chp, boiler, battery, buffer, ee, prices
Outputs:
  obs_next – next observation
  reward – scalar reward
  terminated – episode flag (after 24 steps)
  truncated – always False here
  info – diagnostic dictionary (flows, SOC, costs, etc.)
1. Preprocess action and time index

  1.1 Map raw action from ([-1,1]) to ([0,1]):
      action_raw ← clip(action, -1, 1)
      action_01 ← 0.5 · (action_raw + 1) and clip to ([0,1]`.

  1.2 Compute absolute time index:
      t ← start_hour + episode_t
      If t ≥ n_hours: return zero observation, reward 0, terminated = True.

  1.3 Read current requirements and PV:
      elec_req -> elec_demand[t]
      heat_req -> heat_demand[t]
      pv_available -> max(0, pv_gen[t])

  1.4 Store start-of-step SOC for logging / feasibility layer:
```

```
      soc_elec_start -> soc_batt
      soc_heat_start -> soc_buff

2. Interpret CHP and boiler actions

  2.1 CHP (action[0]):
      If a_chp < 0.05:
        chp_frac = 0, chp_on = 0
      Else:
        chp_frac = 0.6 + 0.4 · a_chp (so 60–100%),
        chp_on = 1
      chp_in = chp_frac · chp.rated_input_heat

  2.2 Boiler (action[3]):
      If a_boiler < 0.05:
        boiler_frac = 0, boiler_on = 0
      Else:
        boiler_frac = 0.2 + 0.8 · a_boiler (20–100%), boiler_on = 1
      boiler_out_total = boiler_frac · boiler.rated_output_heat

3. Electricity balance and storage

  3.1 Compute electricity from CHP and battery:
      chp_total_elec = chp_in · (rated_output_elec / rated_input_heat)
      batt_discharge = clip(action[1] · min(batt.max_discharge_rate_elec,
soc_batt), 0, min(...))

  3.2 Total produced electricity:
      elec_produced = chp_total_elec + pv_available + batt_discharge
      remaining_elec_demand = elec_req − elec_produced

  3.3 Grid and excess:
```

```
      If remaining_elec_demand < 0:
        grid = 0, excess_elec = −remaining_elec_demand
      Else:
        grid = remaining_elec_demand, excess_elec = 0

  3.4 Allocate excess to electric heater and battery:
      remaining_excess -> excess_elec
      ee_in_actual = clip(remaining_excess, 0, ee.rated_input_elec)
      remaining_excess -> remaining_excess − ee_in_actual

  3.5 Update battery SOC:
      Discharge: soc_batt -> soc_batt − batt_discharge /
batt.eta_discharge_elec
      Charge candidate: input_elec = clip(remaining_excess, 0,
batt.max_charge_rate_elec)
      to_store = input_elec · batt.eta_charge_elec
      available_capacity = batt.capacity − soc_batt
      actual_charge_batt = min(to_store, available_capacity)
      soc_batt ← soc_batt + actual_charge_batt

  3.6 PV feed-in value (revenue basis):
      feed_in_value = remaining_excess − actual_charge_batt /
batt.eta_charge_elec

4. Heat balance and buffer

  4.1 Buffer discharge (action[2]):
      buff_discharge = clip(action[2] · soc_buff ·
buffer.eta_discharge_heat, 0, soc_buff · buffer.eta_discharge_heat)

  4.2 Heat from CHP and electric heater:
      chp_out_heat = chp_in · (rated_output_heat / rated_input_heat)
```

ee_out_heat = ee_in_actual · (ee.rated_output_heat / ee.rated_input_elec)

4.3 Pre-boiler total heat and demand:

total_heat_produced = chp_out_heat + ee_out_heat + buff_discharge

remaining_heat_demand = heat_req − total_heat_produced

If remaining_heat_demand < 0: excess_heat = −remaining_heat_demand, unmet_heat = 0

Else: excess_heat = 0, unmet_heat = remaining_heat_demand

Store unmet_heat_pre_boiler = unmet_heat

4.4 Update buffer SOC after discharge:

soc_buff ← soc_buff − buff_discharge / buffer.eta_discharge_heat

buffer_free_capacity = buffer.capacity − soc_buff

4.5 Boiler allocation:

Direct to demand:

boiler_out_dh = min(max(0, remaining_heat_demand), boiler_out_total)

Headroom:

boiler_headroom = max(0, boiler_out_total − boiler_out_dh)

To buffer (action[4]):

boiler_to_buffer = min(boiler_headroom · action[4], buffer_free_capacity)

4.6 Enforce 20% min-load:

If boiler_out_dh + boiler_to_buffer ≤ 0.2 · boiler.rated_output_heat:

boiler_out_dh = 0, boiler_to_buffer = 0, boiler_on = 0

4.7 Update unmet and boiler input:

unmet_heat ← unmet_heat − boiler_out_dh

unmet_heat_post_boiler = unmet_heat

boiler_out = boiler_out_dh + boiler_to_buffer

boiler_in = boiler_out · (boiler.rated_input_heat / boiler.rated_output_heat)

4.8 Excess heat to buffer and waste:

remaining_excess_heat = excess_heat + boiler_to_buffer

Charge candidate:

input_heat = max(0, remaining_excess_heat)

to_store = input_heat · buffer.eta_charge_heat

available_capacity = buffer.capacity − soc_buff

actual_charge_heat = min(to_store, available_capacity)

soc_buff ← soc_buff + actual_charge_heat

Waste:

waste_heat = remaining_excess_heat − actual_charge_heat / buffer.eta_charge_heat

5. Cost and reward

5.1 Economic terms:

gas_cost = (chp_in + boiler_in) · gas_price

elec_cost = grid · grid_price

elec_total = elec_cost − feed_in_value · feed_in_price

5.2 Penalties:

unmet_heat_penalty = 1000 · $unmet\_heat^2$

waste_heat_penalty = 2 · waste_heat

5.3 Switching cost:

chp_switched = 1 if chp_on ≠ prev_chp_on else 0

boiler_switched = 1 if boiler_on ≠ prev_boiler_on else 0

switch_cost = chp_switch_cost · chp_switched + boiler_switch_cost · boiler_switched

```
  5.4 Reward:
      reward = −(gas_cost + elec_total + unmet_heat_penalty +
waste_heat_penalty + switch_cost)

  5.5 Update previous on/off for next step:
      prev_chp_on ← chp_on, prev_boiler_on ← boiler_on

6. Advance time and build outputs

  6.1 Update episode_t -> episode_t + 1, hour ← start_hour + episode_t.
  6.2 terminated -> (episode_t ≥ lookahead), truncated ← False.
  6.3 obs_next -> _get_obs() if not terminated, else zeros.
  6.4 Build info dict with:
      time indices, raw and mapped actions, on/off and switch flags, flows,
SOCs, costs, penalties, start-of-step SOC (if you include them), etc.
  6.5 Return (obs_next, reward, terminated, truncated, info).
```

To ensure that the proposed LP and RL schedules satisfy physical constraints under realized demands, two feasibility layers are employed, both implemented with `cvxpy` in the Optimization module. The function feasibility_layer_LP takes a planned LP schedule and the actual electricity, heat, and PV time series, and solves a single-step LP correction problem that minimally adjusts the planned actions so that all constraints (balances, storage limits, minimum loads) are satisfied; this layer uses L1 penalties or similar terms to keep the adjusted actions close to the original plan while ensuring feasibility. The function `feasibility_layer_RL` performs an analogous correction for the RL-generated actions, again relying on `cvxpy` to enforce the same physics and operating constraints while penalizing deviations from the RL policy's suggested actions. In both cases, the optimization is posed as a convex problem and is solved with numerical solvers accessed through

`cvxpy`, and status information is recorded so that infeasibilities or solver failures can be analyzed.

```
Algorithm : Feasibility correction layer for LP and RL schedules
---------------
Inputs:

  plan – one-step schedule from upstream method (LP or RL)
  elec_real, heat_real, pv_real – realized electricity, heat, and PV for
this step
  components – technical parameters (CHP, Boiler, Battery, Buffer, EE, PV)
  previous_soc, previous_soc_buff – SOC of battery and buffer at end of
previous step (for chaining)
  lambda_res – weight for “resemble plan” penalty
  Output:

Feasible corrected schedule (flows, updated SOC, costs, status)
1. Read inputs and states

  1.1 Extract real demands and PV capacity for this step:
      elec_d -> max(0, elec_real)
      heat_d -> max(0, heat_real)
      pv_cap -> max(0, min(pv_real, PV.capacity))

  1.2 Determine start-of-step SOC:
      If previous_soc / previous_soc_buff are provided in plan, then
        soc_elec -> previous_soc
        soc_heat -> previous_soc_buff
      Else
        soc_elec -> plan.soc_elec
        soc_heat -> plan.soc_heat

2. Define decision variables (all ≥ 0 unless stated)
```

```
  2.1 Heat-side flows:
      chp_hd, boiler_hd, buffer_to_discharge_hd, ee_hd

  2.2 Electricity-side flows:
      pv_ed, battery_to_discharge_ed, chp_ed, grid_out

  2.3 Cross-couplings and storage:
      chp_batt, pv_batt, chp_buff, boiler_buff
      chp_ee, pv_ee, batt_to_discharge_ee
      batt_to_charge, buff_to_charge, ee_ed
      batt_to_discharge_total, chp_heat_total, chp_elec_total, chp_in
      soc_batt_next, soc_buff_next

  2.4 Feed-in to grid:
      pv_feed_in, chp_feed_in

  2.5 Binary on/off for minimum-load behavior:
      chp_on ∈ {0,1}, boiler_on ∈ {0,1}

  (For feasibility_layer_RL, the same structure is used, but starting from
RL actions instead of an LP row.)

3. Add physical and operational constraints

  3.1 Energy balances (no unmet here):
      Heat:
        chp_hd + boiler_hd + buffer_to_discharge_hd + ee_hd = heat_d
      Electricity:
        pv_ed + battery_to_discharge_ed + chp_ed + grid_out = elec_d

  3.2 Linking of flows:
```

Battery charge:

chp_batt + pv_batt = batt_to_charge

Buffer charge:

chp_buff + boiler_buff = buff_to_charge

Electric heater:

pv_ee + chp_ee + batt_to_discharge_ee = ee_ed

3.3 PV limit:

pv_ed + pv_ee + pv_batt ≤ pv_cap

3.4 Boiler with minimum load (0 or 20–100% rated heat):

boiler_out_total = boiler_hd + boiler_buff

boiler_out_total ≤ boiler_on · boiler.rated_output_heat

boiler_out_total ≥ 0.2 · boiler.rated_output_heat · boiler_on

3.5 Electric heater:

ee_ed ≤ ee.rated_input_elec

ee_hd = ee_ed · (ee.rated_output_heat / ee.rated_input_elec)

3.6 Battery constraints:

batt_to_discharge_total = batt_to_discharge_ee + battery_to_discharge_ed

batt_to_discharge_total ≤ soc_elec · battery.eta_discharge_elec

batt_to_discharge_total ≤ battery.max_discharge_rate · Δt · battery.eta_discharge_elec

batt_to_charge ≤ (battery.capacity − soc_elec) · battery.eta_charge_elec

3.7 Buffer constraints:

buffer_to_discharge_hd ≤ soc_heat · buffer.eta_discharge_heat

buff_to_charge ≤ (buffer.capacity − soc_heat) · buffer.eta_charge_heat

```
  3.8 CHP coupling and minimum load (0 or 60–100% rated input):
      chp_in ≤ chp_on · chp.rated_input_heat
      chp_in ≥ 0.6 · chp.rated_input_heat · chp_on
      chp_heat_total = chp_in · (chp.rated_output_heat /
chp.rated_input_heat)
      chp_elec_total = chp_in · (chp.rated_output_elec /
chp.rated_input_heat)
      chp_heat_total ≥ chp_hd + chp_buff
      chp_elec_total ≥ chp_ed + chp_ee + chp_batt

  3.9 SOC update (one-hour step):
      soc_batt_next = soc_elec + batt_to_charge · battery.eta_charge_elec −
batt_to_discharge_total / battery.eta_discharge_elec
      soc_buff_next = soc_heat + buff_to_charge · buffer.eta_charge_heat −
buffer_to_discharge_hd / buffer.eta_discharge_heat
      0 ≤ soc_batt_next ≤ battery.capacity
      0 ≤ soc_buff_next ≤ buffer.capacity

  3.10 Feed-in to grid:
      pv_feed_in ≤ pv_cap − (pv_ed + pv_ee + pv_batt)
      chp_feed_in ≤ chp_elec_total − (chp_ed + chp_ee + chp_batt)

(For feasibility_layer_RL, the structure is very similar; the main
difference is which plan fields are used in the resemblance term.)

4. Define objective

  4.1 Operating cost:
      cost = chp_in · gas_price + boiler_hd · gas_price + grid_out ·
grid_price − (pv_feed_in + chp_feed_in) · feed_in_price

  4.2 Resemblance to original plan:
```

```
      Form a vector of key flows (e.g. chp_hd, boiler_hd,
buffer_to_discharge_hd, battery_to_discharge_ed, grid_out, pv_ed, pv_ee,
pv_batt)
      Subtract the corresponding values from plan (or 0 if missing/NaN).
      resemble = λ_res · L1_norm( current_flows − plan_flows )

  4.3 Minimize total objective:
      minimize cost + resemble

5. Solve and return

  5.1 Call a MIP/LP solver (e.g. GUROBI via CVXPY) on the defined problem.
  5.2 If a solution is found, extract all relevant scalar values (flows,
SOC, on/off flags, prices).
  5.3 If no solution is found, return safe defaults and status "infeasible"
or "error".
  5.4 Return a record containing:
      corrected flows (boiler, Grid, chp_out_*, pv_out_*, buffer_out,
batt_out_*, ee_heat),
      updated SOC (soc_elec, soc_heat),
      prices (gas_price_*, elec_price_*),
      and meta-information (status, solver_used, solver_verbose).
```

Constraint satisfaction is further checked by a constraint checker in `Optimization/constraint_checker.py`, which is used to verify, for each time step and each constraint (e.g. power balance, SOC limits, non-negativity), whether the resulting LP or RL schedule satisfies the desired tolerances. This checker is often applied to the outputs of the feasibility layers to quantify residual violations.

Finally, a baseline schedule (for example, a simple rule-based or naive allocation) is constructed, and all three approaches baseline, LP (with or

without switching cost), and RL (possibly post-processed by the feasibility layer) are evaluated on a common price-based cost metric. This evaluation is typically orchestrated in analysis notebooks such as `optimizer_comparisons.ipynb`, where costs, switching behavior, and constraint violations are compared. In this way, the optimization pipeline moves from demand and component data, through LP and RL scheduling (using `cvxpy` and `gymnasium`), to feasibility repair and constraint checking, and finally to a comparison of competing control strategies based on economic criteria.

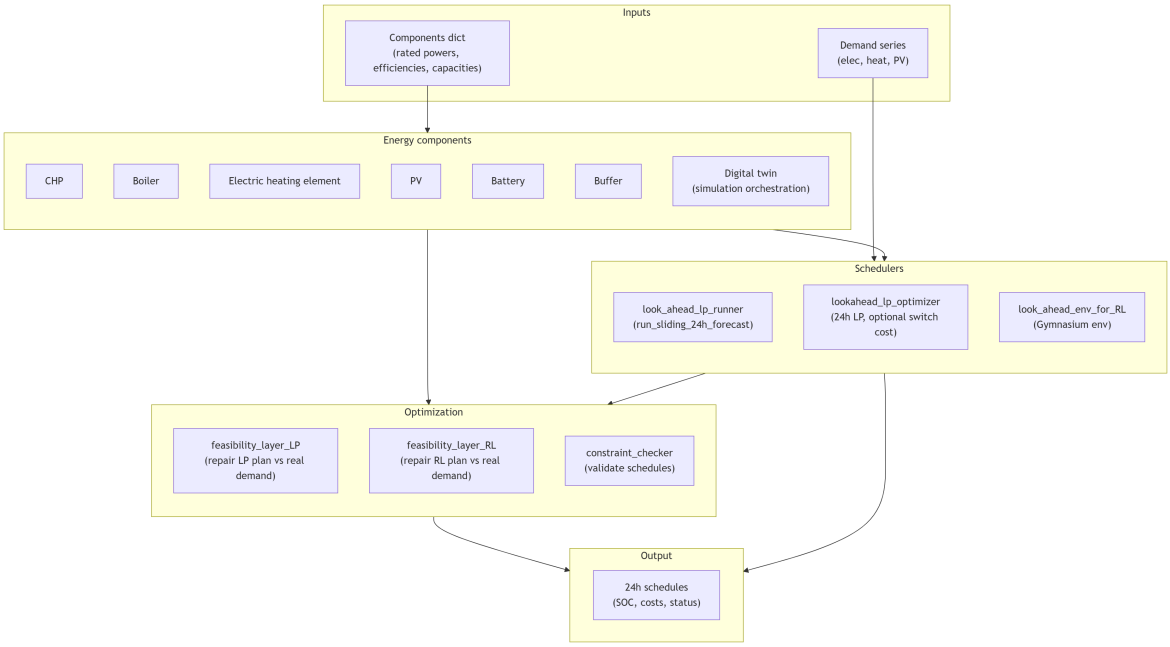

Figure 12: Evaluation methodology for different schedulers.

7. Results

In the results section, a more detailed analysis of the intermediate results from the forecasting and optimization steps is presented, providing additional background and credibility for the final results.

7.1 Electricity Demand Forecast

This section evaluates the performance of different forecasting approaches for electricity load, including SARIMA, XGBoost, and neural network models. The objective is to compare how model complexity, preprocessing choices, and training configurations affect predictive accuracy and generalization to out-of-sample data. Results show that simpler or moderately complex models often perform best because they capture the dominant 24-hour demand cycle while avoiding overfitting. In particular, the SARIMA model (0,0,1)(0,1,1,24), a well-configured XGBoost pipeline with appropriate preprocessing and external features, and a moderately sized neural network architecture provide the strongest validation performance. The analysis also examines residual behavior and training dynamics to assess model assumptions, overfitting risks, and the effectiveness of early stopping during training. Similar processes has been implemented for heat and PV as well

7.1.1 SARIMA model

The best results for the SARIMA was the one with the simplest model with order (0,0,1) and seasonal order (0,1,1,24). As we increase the complexity of the model, we see a clear decrease in the training loss, but cannot generalize to out-of-sample datasets, leading to overfitting. Electricity load is dominated

by 24 hr cycle and the seasonal order of (0,1,1,24) was enough to outperform other more complex structures.

pipeline_idx	order	seasonal_order	rmse	mbe	elapsed_sec	train_rmse	train_mbe
2	(0, 0, 1)	(0, 1, 1, 24)	15.40	-0.07	9.19	11.06	-0.07
2	(0, 0, 1)	(1, 1, 1, 24)	15.45	0.22	13.58	10.58	0.19
2	(1, 0, 0)	(1, 1, 1, 24)	15.46	-0.18	12.01	8.05	-0.00
2	(1, 0, 1)	(1, 1, 1, 24)	15.47	-0.19	16.70	8.04	-0.00
2	(1, 0, 0)	(0, 1, 1, 24)	15.47	-0.27	5.68	8.20	-0.03

Table 8: Summary results of SARIMA electricity Forecast

The residuals are not well approximated by a normal distribution, as a normal curve would be symmetric and bell-shaped, whereas the observed histogram shows a clear positive skew with a long right tail. This indicates that while most residuals are close to zero, there are occasional large positive errors. The Shapiro–Wilk test confirms this by showing p-values of 6.8×10^{-37}, which leads us to reject the null hypothesis of normal distribution.

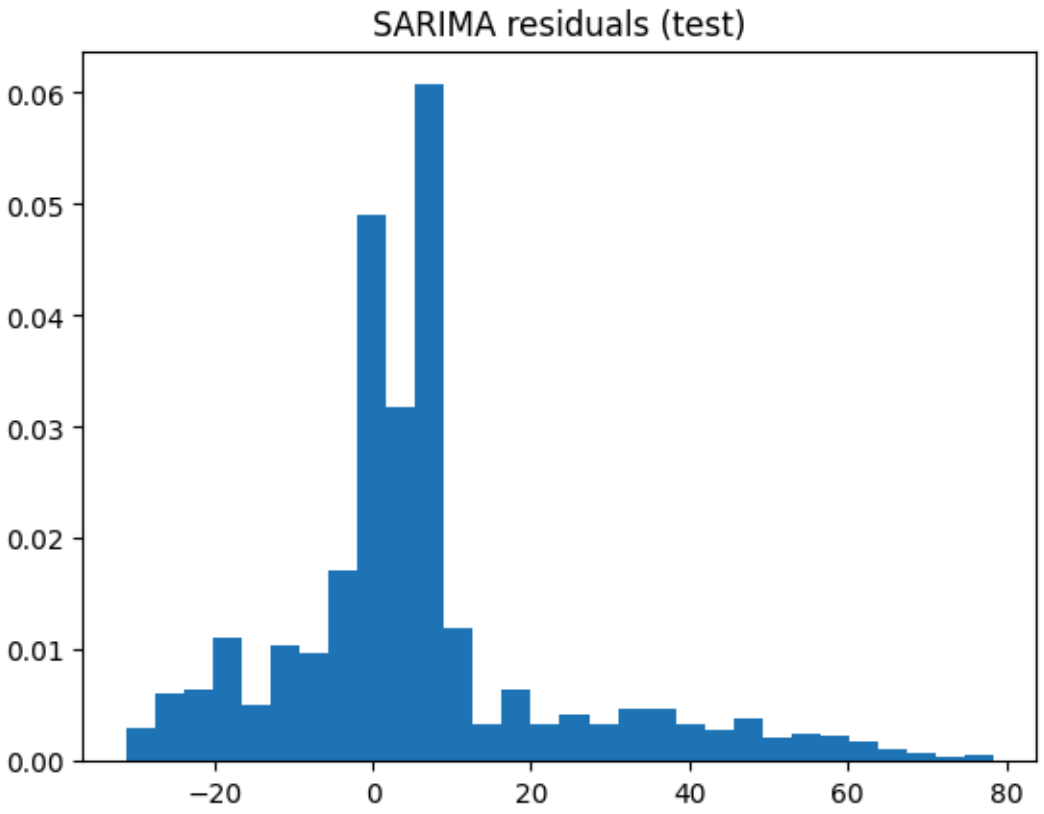

Figure 13: Residual plot distribution of test set of electricity demand from SARIMA forecasts with best hyper parameters

7.1.2 XgBoost model

From the table Table 9, the preprocessing step with pipeline index 1 has consistently shown lower rmse values than the next-highest pipeline index 2. The first pipeline implemented a multitude of preprocessing steps, including standardization and the addition of external variables such as calendar and holiday features. Secondly, the maximum depth increases the model's complexity, leading it to overfit the training data but not generalize to out-of-sample datasets. The same goes for the lag features. As the number of lag features increases, the gap between the training and validation data widens, indicating overfitting.

pipeline_idx	n_estimators	max_depth	rmse	mbe	elapsed_sec	train_rmse	train_mbe
2	50	3	11.10	0.60	0.02	10.40	-0.00
1	100	3	6.00	0.37	0.03	4.69	-0.00
1	50	3	6.14	0.52	0.16	5.09	-0.00
1	50	6	6.33	0.13	0.04	3.06	-0.00
1	100	6	6.59	0.16	0.05	2.31	-0.00

Table 9: Summary results of Xgboost electricity Forecast

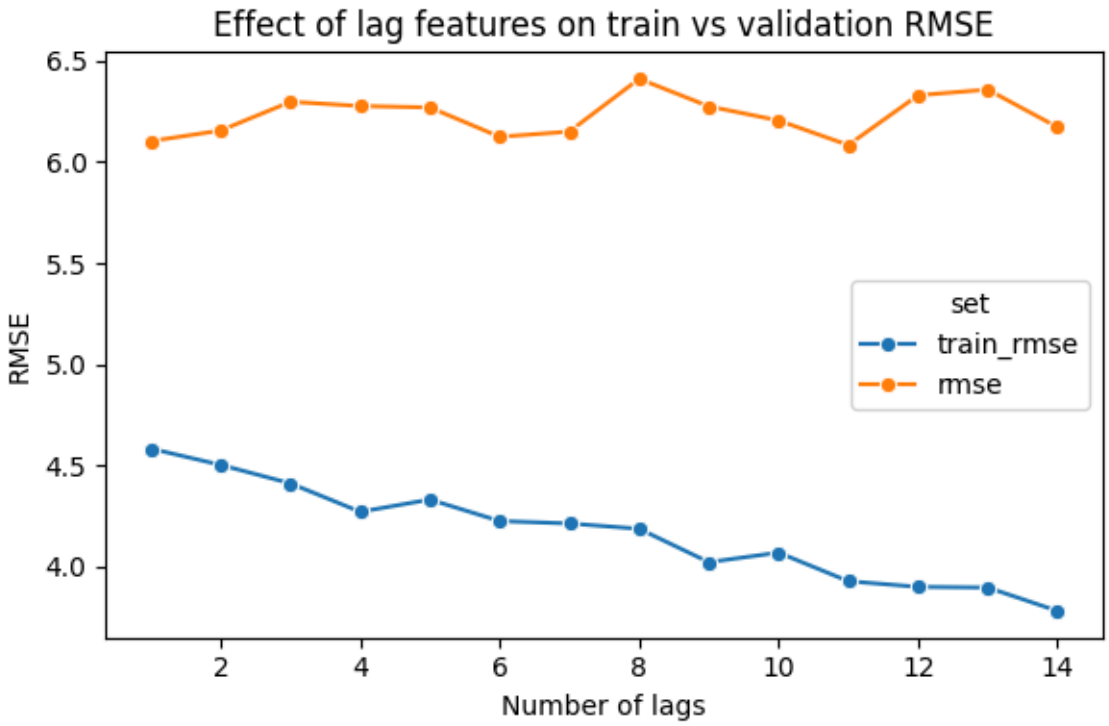

Figure 14: Effect of Lag features on train and validation scores for electricity forecast

7.1.3 LSTM model

The best validation/test RMSE is 7.404, achieved with a moderate architecture (32 hidden units, 2 layers, 50 epochs, patience 5, dropout 0.1, learning rate 0.001). Increasing model complexity (e.g., 4 layers) or training longer (100 epochs) does not improve test performance. Training RMSE remains consistently below validation RMSE, but the gap is small, indicating no strong overfitting across configurations.

pipeline_idx	hidden_size	num_layers	epochs	patience	dropout	lr	rmse	mbe	elapsed_sec	train_rmse	train_mbe
1	32	2	50	5	0.100	0.001	7.404	-0.169	3.556	6.353	-0.681
1	16	2	100	5	0.100	0.001	7.450	0.952	4.174	6.719	-0.183
1	32	2	100	15	0.100	0.001	7.488	0.028	7.708	5.590	0.238
1	16	4	50	15	0.100	0.001	7.507	0.972	14.786	5.945	-0.189
1	16	4	100	5	0.100	0.001	7.584	1.162	5.106	7.371	0.054

Table 10: Summary results of LSTM electricity Forecast

The training loss decreases rapidly from about 0.65 to about 0.17 within the first two epochs, then gradually declines to around 0.09–0.10. Validation loss follows a similar pattern, dropping from about 0.22 to about 0.14–0.15 early on and stabilizing near 0.10–0.11, closely tracking the training curve. Since validation loss does not rise while training loss decreases, there is no clear overfitting, and most learning occurs in the first 8–10 epochs, suggesting that early stopping around 8–12 epochs would be sufficient.

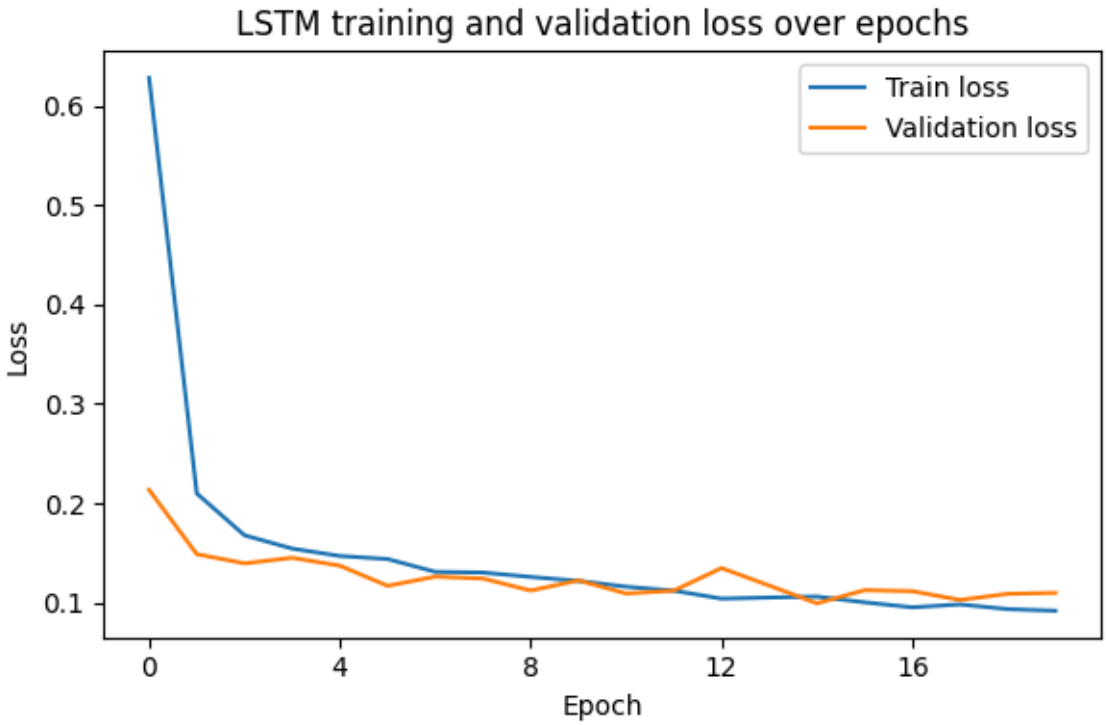

Figure 15: LSTM training and validation loss over epochs with best performing hyper-parameter for electricity

7.2 Heat Demand Forecast

7.2.1 SARIMA model

pipeline_idx	order	seasonal_order	rmse	mbe	elapsed_sec	train_rmse	train_mbe
1	(1, 0, 0)	(1, 1, 1, 24)	12.11	-1.19	6.49	15.65	0.36
2	(0, 0, 1)	(1, 1, 1, 24)	12.12	-0.66	6.25	13.98	0.91
2	(0, 0, 0)	(1, 1, 1, 24)	12.19	-0.68	4.88	15.24	1.30
1	(1, 0, 0)	(0, 1, 1, 24)	12.22	-1.94	3.38	14.62	-0.16
1	(1, 0, 1)	(0, 1, 1, 24)	12.34	2.55	14.65	176.47	2.48

Table 11: Summary results of SARIMA heat Forecast

The best test RMSE (12.11) is achieved by the SARIMA model with order (1,0,0)(1,1,1,24) in pipeline 1. This structure combines a non-seasonal AR(1) term to capture short-term persistence with a seasonal (1,1,1,24) component that models the dominant 24-hour cycle through one seasonal difference, a seasonal AR term, and a seasonal MA term. Alternative specifications perform slightly worse: removing the seasonal AR term (0,1,1,24) increases RMSE slightly to 12.22, while adding a non-seasonal MA term (1,0,1)(0,1,1,24) produces unstable results with a very large training RMSE. Overall, the combination of short-term autoregression and daily seasonal structure provides the most accurate forecasts for the heat series.

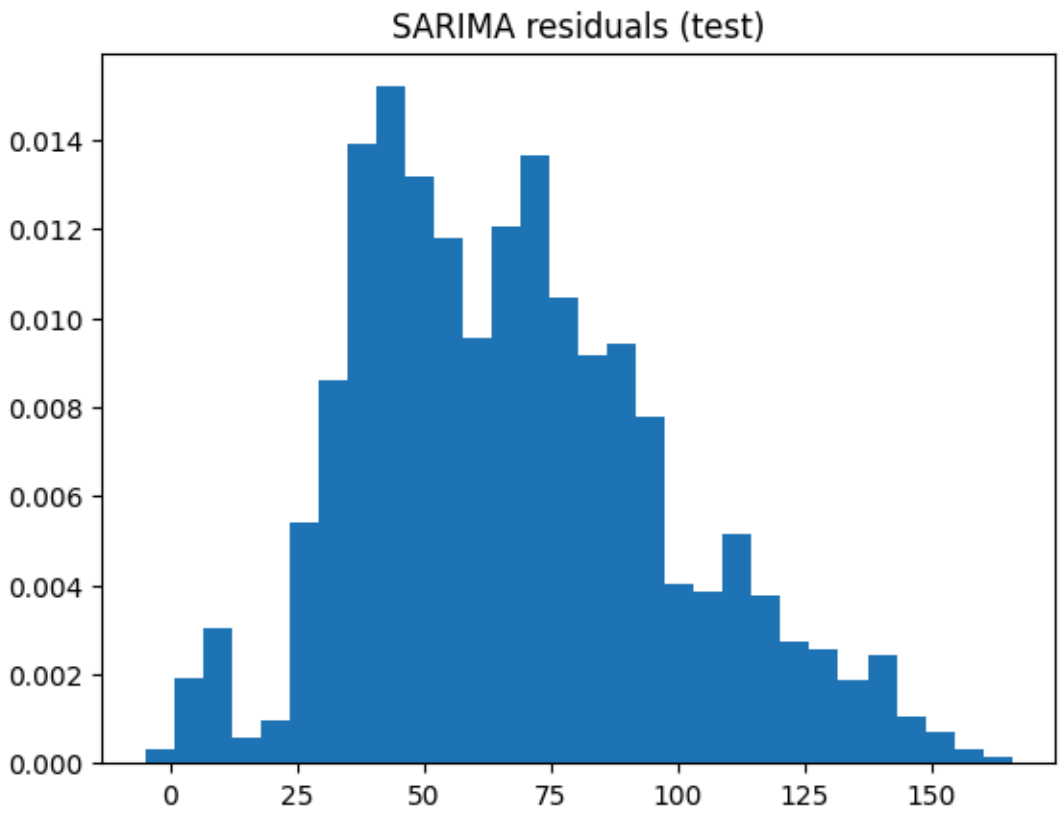

Figure 16: Residual plot distribution of test set of heat demand from SARIMA forecasts with best hyper-parameters

The residual histogram for the SARIMA heat model is strongly right-skewed, with a main concentration around 40–50, additional peaks near 70–80 and 110–115, and a long tail extending to roughly 170. This indicates that residuals are not normally distributed, as they do not form a symmetric bell-shaped pattern around zero. The dominance of large positive residuals suggests the model tends to underpredict heat demand during certain periods, which is consistent with the slightly negative mean bias error (MBE ≈ −1.19). While the selected SARIMA specification performs well for point forecasting (low RMSE), the skewed residual distribution implies that prediction intervals based on normality assumptions may be unreliable. This can also be confirmed by the Shapiro–Wilk test with 1.3×10^{-17} rejecting the normality assumption of the residuals.

7.2.2 XgBoost model

The best test RMSE (7.60) is achieved with pipeline 2, 100 estimators, and max_depth = 3, with a training RMSE of 6.87 and MBE of 1.21. Pipeline 2 consistently outperforms pipeline 1 (e.g., 7.60 vs 7.90 and 7.64 vs 8.04), suggesting that its feature set likely including improved lag or external variables better captures the heat demand dynamics. Increasing tree depth to 6 reduces training RMSE substantially (4.40) but worsens test RMSE (7.97), indicating clear overfitting. The number of estimators has only a minor impact: 100 estimators slightly improves RMSE compared to 50, although 50 estimators train faster.

pipeline_idx	n_estimators	max_depth	rmse	mbe	elapsed_sec	train_rmse	train_mbe
2	100	3	7.60	1.21	0.06	6.87	0.00
2	50	3	7.64	1.84	0.03	7.67	-0.00
1	50	3	7.90	2.57	0.04	7.06	-0.00
2	50	6	7.97	0.76	0.06	4.40	-0.00
1	100	3	8.04	2.23	0.07	6.27	0.00

Table 12: Summary results of Xgboost heat Forecast

The lag-feature analysis shows that training RMSE steadily decreases as the number of lags increases, dropping from about 7.0 at 1 lag to 6.3 at 12–14 lags, indicating improved fit on the training data. Validation RMSE decreases sharply at first, falling from roughly 11.5 at 1 lag to around 8–8.3 by 8–14 lags, before stabilizing. Although validation error remains consistently above training error indicating some overfitting it still improves as more lag features are added. The lowest validation RMSE occurs around 12–14 lags, though gains beyond roughly 8 lags are small, making 8–12 lags a practical trade-off between predictive performance and model simplicity.

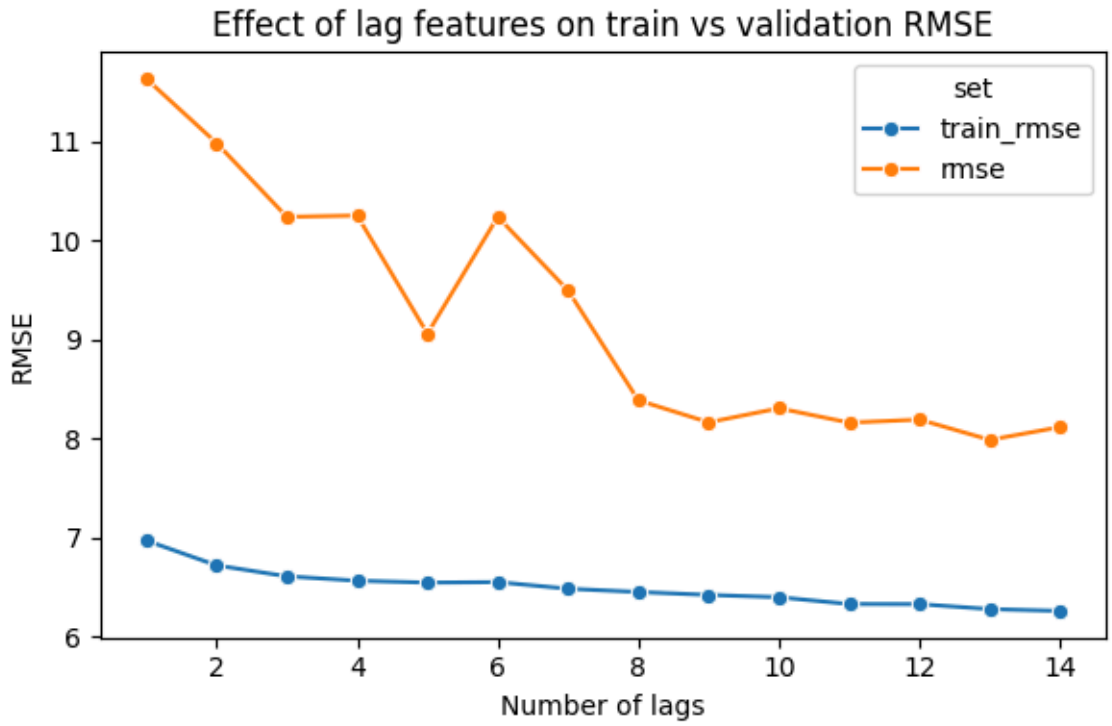

Figure 17: Effect of Lag features on train and validation scores for heat forecast

7.2.3 LSTM model

The best test RMSE (8.945) is achieved with pipeline 1, hidden size 16, 2 layers, 50 epochs, patience 15, dropout 0.1, and learning rate 0.001, with a training RMSE of 8.103 and MBE of 1.451. Increasing model depth to 4 layers slightly lowers training RMSE (7.750) but worsens test RMSE (9.244), indicating overfitting due to excess capacity. Extending training to 100 epochs also does not improve performance and sometimes degrades it. Additionally, patience 15 performs better than patience 5, suggesting that allowing the model more time before early stopping helps achieve better generalization. Overall, the small gap between training and test RMSE indicates limited overfitting in the best configuration.

pipeline_idx	hidden_size	num_layers	epochs	patience	dropout	lr	rmse	mbe	elapsed_sec	train_rmse	train_mbe
1	16	2	50	15	0.100	0.001	8.945	1.451	10.458	8.103	-0.373
1	16	4	100	15	0.100	0.001	9.244	1.759	23.172	7.750	0.263
1	16	2	100	5	0.100	0.001	9.280	4.222	5.765	8.414	-0.101
1	16	2	50	5	0.100	0.001	9.388	3.430	3.216	8.922	0.665
1	16	2	100	15	0.100	0.001	9.431	2.734	14.511	7.833	0.145

Table 13: Summary results of LSTM heat Forecast

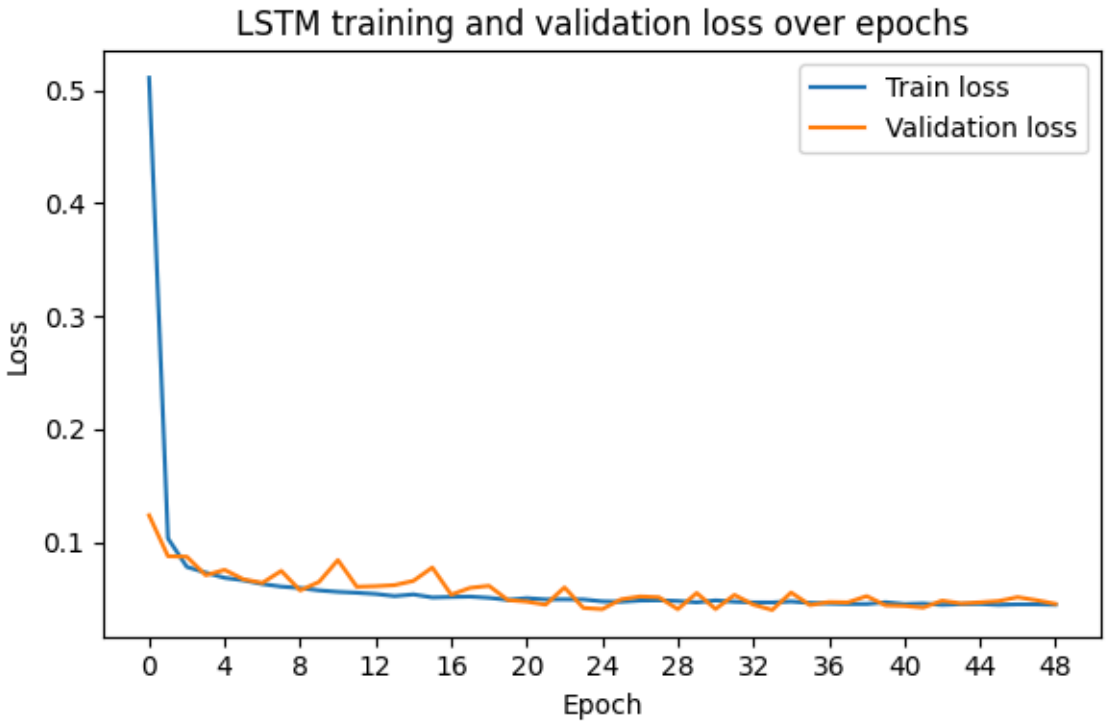

Figure 18: LSTM training and validation loss over epochs with best performing hyper-parameter for heat

The training and validation loss curves show rapid learning in the first few epochs. Training loss begins above 0.5, while validation loss starts around 0.12, and both drop quickly below 0.1 within the first about 4 epochs. After roughly 8–10 epochs, the curves flatten, with training loss stabilizing and validation loss fluctuating slightly while closely tracking the training curve. Because validation loss does not increase while training loss decreases, no clear overfitting is observed. Most learning occurs within the first 10–20 epochs, and later improvements are small, supporting the effectiveness of early stopping with moderate patience, as used in the best-performing run.

7.3 PV generated Forecast

7.3.1 SARIMA model

The best test RMSE (9.95) is obtained in pipeline 2 for four SARIMA specifications: (1,0,0) or (1,0,1) combined with (0,1,1,24) or (1,1,1,24). All four configurations produce the same MBE (3.73) and train RMSE (4.34). The results indicate that the seasonal component (0,1,1,24) is already sufficient to capture the dominant 24-hour PV cycle, since adding a seasonal AR term does not improve performance. Similarly, the AR(1) term captures short-term persistence, and adding a non-seasonal MA(1) does not reduce test error. The relatively low training RMSE compared to the test RMSE suggests a stronger in-sample fit or a more challenging test period. The positive MBE indicates that the model systematically overpredicts PV generation.

pipeline_idx	order	seasonal_order	rmse	mbe	elapsed_sec	train_rmse	train_mbe
1	(1, 0, 1)	(1, 1, 1, 24)	10.06	3.49	12.61	4.37	-0.51
2	(1, 0, 0)	(0, 1, 1, 24)	9.95	3.73	6.52	4.34	-0.25
2	(1, 0, 1)	(0, 1, 1, 24)	9.95	3.73	7.89	4.34	-0.25
2	(1, 0, 0)	(1, 1, 1, 24)	9.95	3.73	12.32	4.34	-0.25
2	(1, 0, 1)	(1, 1, 1, 24)	9.95	3.73	15.63	4.34	-0.25

Table 14: Summary results of SARIMA pv Forecast

The residual histogram shows a highly skewed distribution bounded above by zero, meaning residuals are either zero or negative. This implies the model never underpredicts in the test set and instead tends to overpredict PV output. A strong spike occurs at 0, reflecting many hours with nearly perfect predictions, which likely correspond to nighttime periods when PV generation is zero. Away from zero, residuals extend downward to roughly −50, with smaller concentrations at intervals along the negative range. The distribution is therefore non-normal, with a large mass at zero and a long negative tail.

This pattern is consistent with the zero-inflated nature of PV generation and aligns with the positive MBE, indicating systematic overprediction during certain daylight periods.

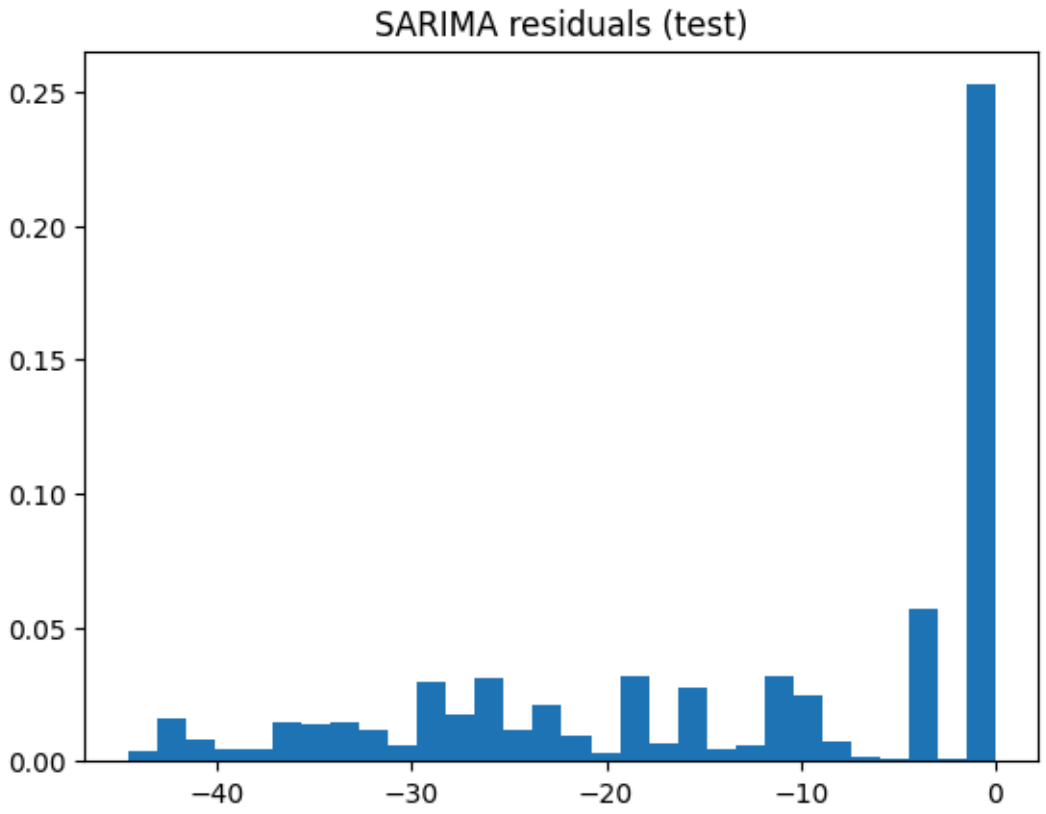

Figure 19: Residual plot distribution of test set of pv demand from SARIMA forecasts with best hyper-parameters

7.3.2 XgBoost model

The best test RMSE (4.75) is achieved with pipeline 1, 50 estimators, and max_depth = 3, with a training RMSE of 3.10 and an MBE of −0.03. Pipeline 1 clearly outperforms pipeline 2 (4.75–5.01 vs 6.75), indicating that its preprocessing or feature set, likely including better lag or input variables, is more suitable for PV forecasting. Increasing tree depth to 6 greatly reduces training RMSE (0.67–1.33) but worsens test RMSE (4.91–5.01), showing clear overfitting. Similarly, increasing the number of estimators from 50 to 100

improves training fit but does not improve generalization, making 50 estimators with shallow trees the best-performing configuration.

pipeline_idx	n_estimators	max_depth	rmse	mbe	elapsed_sec	train_rmse	train_mbe
1	50	3	4.75	-0.03	0.19	3.10	-0.00
1	100	3	4.90	0.08	0.04	2.65	-0.00
1	50	6	4.91	0.15	0.04	1.33	-0.00
1	100	6	5.01	0.15	0.07	0.67	-0.00
2	50	3	6.75	-0.30	0.02	5.08	-0.00

Table 15: Summary results of Xgboost PV Forecast

The lag-feature analysis shows that training RMSE remains consistently low, starting around 2.7 at 1 lag and reaching its lowest point near 2.25–2.3 at around 12 lags, before slightly increasing again. Validation RMSE, however, starts higher at about 4.9, increases slightly at 3 lags, then gradually decreases to its minimum of about 4.58–4.6 at 12 lags, before rising again at 14 lags. The persistent gap between training and validation RMSE indicates clear overfitting or a more challenging validation period, where the model fits training data much better than unseen data. Overall, 12 lag features provide the best trade-off, as too few or too many lag inputs worsen validation performance.

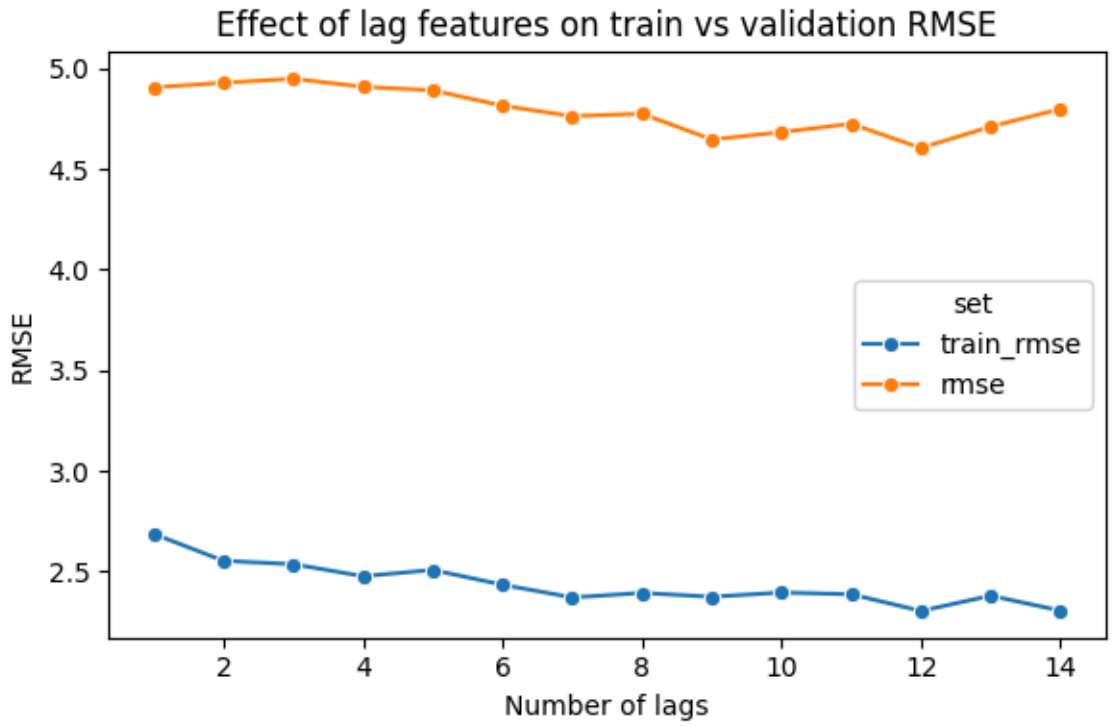

Figure 20: Effect of Lag features on train and validation scores for PV forecast

7.3.3 LSTM model

The best test RMSE (5.174) is achieved with pipeline 1, hidden size 16, 2 layers, 100 epochs, patience 15, dropout 0.1, and learning rate 0.001, with a training RMSE of 4.126 and MBE 0.258. Models with 16 hidden units outperform larger models with 32 units, as the larger architecture produces lower training RMSE but slightly worse test RMSE, indicating mild overfitting. Allowing longer training (100 epochs) together with higher patience (15) yields the best results, while shorter training or lower patience performs slightly worse. The moderate gap between training and test RMSE suggests reasonable generalization without strong overfitting.

pipeline_idx	hidden_size	num_layers	epochs	patience	dropout	lr	rmse	mbe	elapsed_sec	train_rmse	train_mbe
1	16	2	100	15	0.100	0.001	5.174	0.258	7.231	4.126	0.180
1	16	2	50	5	0.100	0.001	5.319	-0.308	3.100	4.644	-0.088
1	32	2	50	5	0.100	0.001	5.454	0.257	1.315	4.441	0.028
1	32	2	100	15	0.100	0.001	5.456	-0.197	4.141	4.096	-0.055
1	32	2	100	5	0.100	0.001	5.461	1.113	2.173	4.352	0.620

Table 16: Summary results of LSTM pv Forecast

The training and validation loss curves show rapid improvement in the early epochs. Training loss begins around 0.88 and validation loss near 0.77, with both decreasing sharply by epoch 4. After this initial phase, training loss gradually stabilizes, while validation loss fluctuates. The validation curve remains slightly above the training curve with a stable gap, indicating good generalization and no severe overfitting. Most learning occurs within the first 8–12 epochs, after which improvements are minimal, supporting the use of early stopping with moderate patience.

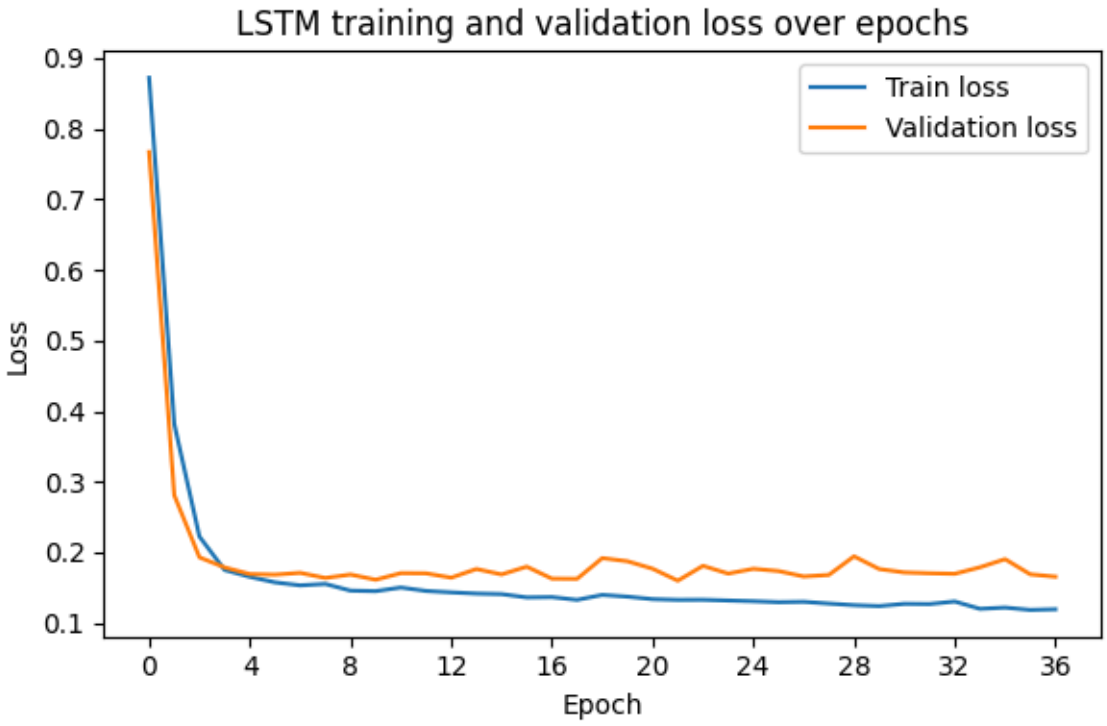

Figure 21: LSTM training and validation loss over epochs with best performing hyper-parameter for pv

7.4 Optimization

This section presents the results of the previously described optimization framework. The objective is to evaluate the transition from rule-based scheduling to more dynamic approaches based on linear programming (LP) and reinforcement learning (RL). To assess their effectiveness, these optimization strategies are compared against a baseline schedule representing current operational guidelines. The evaluation is performed over the final three months of the dataset, consistent with the forecasting test horizon, and uses the forecasted electricity demand, heat demand, and PV generation, together with a parameterized energy system model. For each approach, operating schedules are generated while respecting the physical constraints of system components such as generation units and storage. To ensure feasibility under realized-demand conditions, the schedules are verified and, when necessary, minimally adjusted to satisfy operational constraints. The resulting schedules from the baseline, LP, and RL approaches are then compared using economic performance metrics, allowing a systematic assessment of the benefits and limitations of the proposed optimization strategies. First, the feasibility layer is investigated to determine whether the resulting schedules are feasible and meet the buildings' demand and abide by the physical constraints of the components. From Table 17, The LP is feasible almost everywhere (99.95%). The single infeasible case is 2024-12-09 10:00:00, where heat demand 193 exceeds what the boiler and CHP can jointly supply, causing the solver to report infeasibility and the one "false" result in the feasibility summary. In the LP trajectory, at that time, there was no buffer discharge that could have provided the extra capacity.

Feasiblity check	pct	infeasible_indexes
heat_demand_met	99.95	[Timestamp('2024-12-09 10:00:00')]
elec_demand_met	99.95	[Timestamp('2024-12-09 10:00:00')]
batt_charge_chp_pv	100.00	[]
buff_charge_chp_boil	100.00	[]
elec_to_ee_by_pv_chp_batt	100.00	[]
pv_generated_alloc	100.00	[]
boiler_alloc	100.00	[]
boiler_out_ratings	100.00	[]
boiler_in_out_rln	100.00	[]
ee_elec_ratings	100.00	[]
ee_in_out_rln	100.00	[]
batt_disch_to_de_ee	100.00	[]
batt_disch_ratings	100.00	[]
buff_disch_ratings	100.00	[]
buff_charge_ratings	100.00	[]
batt_charge_ratings	100.00	[]
chp_to_heat_buff	100.00	[]
chp_to_ele_batt_ee	100.00	[]
chp_in_out_rln_heat	100.00	[]
chp_in_out_rln_elec	100.00	[]
chp_out_elec_ratings	100.00	[]
chp_out_heat_ratings	100.00	[]
chp_threshold	100.00	[]
boiler_threshold	100.00	[]

Table 17: Feasibility check for LP optimizer

For Table 18 , we see that almost all values are 100% except for the electricity demand which is at 40%. This is not because of unmet electricity demand but rather a definition and representation of RL. In the RL formulation, there are no explicit “CHP to boiler”, “CHP to battery”, etc. The policy uses high-level quantities (e.g. chp_in, boiler_in, ...). The internal use of electricity (heat, battery, etc.) is not separated from those high-level outputs. The checker thus compares the total electricity generated with the total electricity demand, which might not always be equivalent, as the electricity generated not only supplies the demand but also charges the battery and heats the

heating element. making the generated electricity usually exceed the building's electric demand. A further inspection is required because an invalid state occurs when unmet demand arises, i.e., the total generated electricity cannot meet demand. This can be tested using code:

```
rl_feasible_df_check_result[rl_feasible_df_check_result.elec_demand_met>1]
```

which shows an empty set, confirming there are no invalid solutions.

Feasiblity check	pct
heat_demand_met	100.00
elec_demand_met	40.16
boiler_out_ratings	100.00
boiler_in_out_rln	100.00
ee_elec_ratings	100.00
ee_in_out_rln	100.00
batt_disch_ratings	100.00
buff_disch_ratings	100.00
buff_charge_ratings	100.00
batt_charge_ratings	100.00
chp_in_out_rln_heat	100.00
chp_in_out_rln_elec	100.00
chp_out_elec_ratings	100.00
chp_out_heat_ratings	100.00
chp_threshold	100.00
boiler_threshold	100.00

Table 18: Feasibility check for RL optimizer

The 24-hour LP optimization before feasibility correction achieves the lowest total cost of 31,361, primarily due to the lowest electricity cost (4,931) and a substantially reduced switching cost (150) compared to the baseline value of 1,700. Gas costs are also slightly lower (26,280 vs. 26,654 in the baseline). This indicates that the LP optimizer is able to find a more efficient schedule than the rule-based baseline when no additional feasibility adjustments are required.

When the LP schedule is passed through the feasibility layer, the total cost increases from 31,361 to 33,632. The main reason is a significant rise in electricity costs (4,931 → 6,796) and a slight increase in gas consumption, while switching costs decrease slightly (150 → 100). This indicates that the feasibility correction must adjust the LP plan to satisfy all operational constraints, which requires purchasing additional energy and therefore increases total cost.

A similar effect is observed for the RL-based scheduler. Before feasibility correction, RL achieves a total cost of 33,178, with an extremely low switching cost of only 20. However, after applying the feasibility layer, the total cost increases substantially to 35,185. This increase is largely due to a sharp rise in electricity costs (6,070 → 9,782) and higher switching costs (20 → 390), although gas costs decrease (27,088 → 25,013).

Overall, the results show that optimization-based approaches (LP and RL) can outperform the baseline in terms of total operating cost when feasibility corrections are not applied, with LP before feasibility achieving the best result. However, enforcing feasibility constraints generally increases total cost because schedules must be adjusted to meet all physical and demand requirements. While the RL approach achieves the lowest switching cost before feasibility, it results in the highest total cost after feasibility correction, primarily due to increased electricity consumption and switching behavior.

Method	Gas (€)	Electricity (€)	Switch(€)	Total(€)
Baseline	26,653.60	5,807.90	1,700	34,161.50
Baseline (forecast)	26,419.92	5,421.39	1700	33,541.31
24h LP (before feasibility)	26,279.67	4931.40	150	31,361.07
24h LP (after feasibility)	26,736.14	6,795.53	100	33,631.68
24h RL (before feasibility)	27,088.22	6069.77	20	33,177.99
24h RL (after feasibility)	25,012.65	9,782.07	390	35,184.73

Table 19: Total cost comparisions for baseline, LP and RL schedulers

Figure 22 compares boiler input, CHP input, heat buffer discharge, and battery discharge over October 2024 to end of December for three schedulers: LP, RL, and the baseline. For the boiler, the baseline keeps it almost always on, closely tracking demand, while LP produces short, high peaks with frequent off periods, and RL is smoother with moderate peaks and occasional downtime. This indicates that turning the boiler off when heat can be met by other sources (CHP, buffer, or lower demand), as LP and RL do, is a more cost-effective strategy than keeping it continuously on.

For CHP operation, the baseline runs it nearly at full output throughout, LP mostly at full output with occasional targeted shutdowns, and RL often at reduced levels or fully off for extended periods. This demonstrates that strategic modulation or selective shutdown of CHP, as in LP and RL, can reduce costs compared to constant full-output operation. Heat buffer use differs across schedulers: the baseline discharges steadily, LP discharges rarely but in short high peaks, and RL uses the buffer more frequently and continuously, particularly when CHP is off. This suggests that using the buffer to decouple generation from demand and cover periods when CHP or boiler are off is an effective policy. Battery discharge is minimal for all schedulers, with LP occasionally discharging slightly and RL hardly at all,

indicating that in this setup, the battery primarily serves as a flexible storage or for charging rather than frequent discharge.

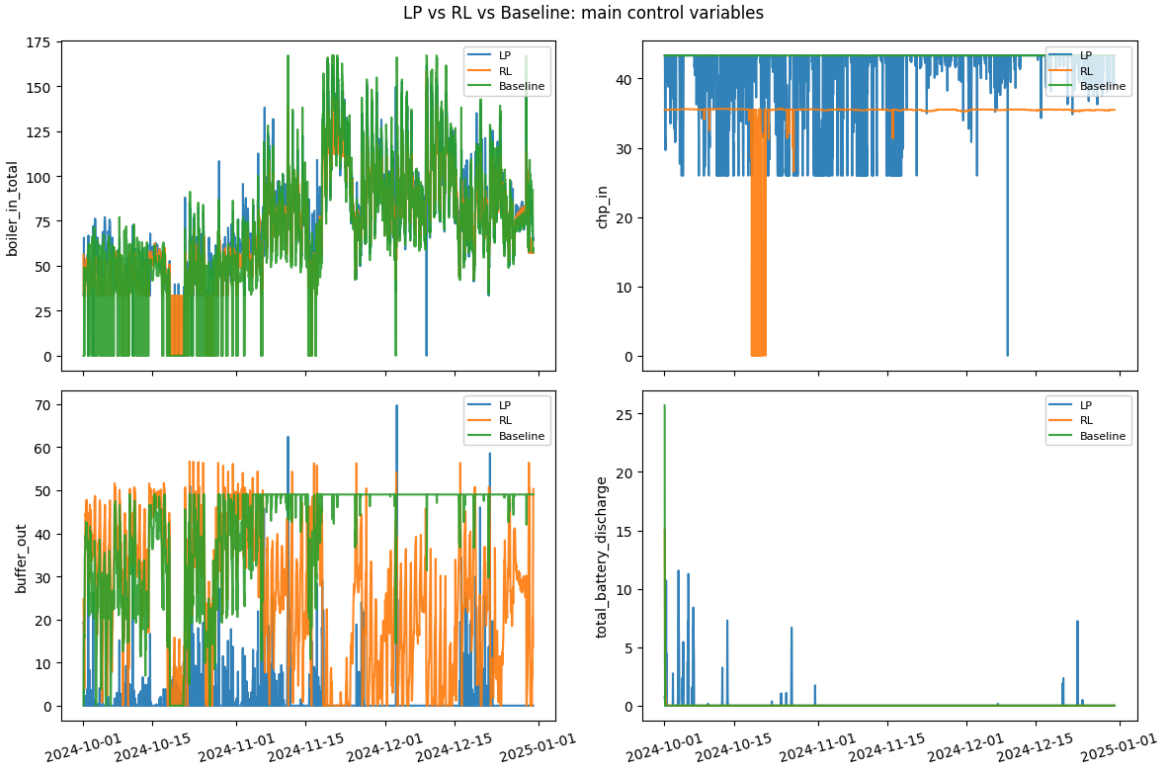

Figure 22: LSTM training and validation loss over epochs with best performing hyper-parameter for pv

Cross-cutting insights show that dynamic, state-dependent control is superior to static operation. LP and RL, which achieve lower total costs, adapt generation and storage usage based on demand, forecasts, and economics. Good policies include running CHP or boiler only when justified by demand or cost, using the heat buffer to shift energy over time, and using batteries primarily for flexibility or charging. LP tends to concentrate boiler and buffer use in short, intense bursts, while RL uses the buffer more continuously and strategically reduces CHP operation. Both approaches outperform the baseline, highlighting the value of flexible, conditional control of energy assets in reducing operating costs and improving system efficiency.

8. Conclusion

The work aimed to design and evaluate a forecasting and scheduling pipeline for an integrated energy system comprising combined heat and power (CHP), boilers, heat buffer, battery, and photovoltaic (PV) generation. The objective was twofold: obtain accurate short-term forecasts of electricity demand, heat demand, and PV generation; and use these forecasts within optimization-based and learning-based schedulers (linear programming and reinforcement learning) to minimize the total cost of gas, electricity, and switching while meeting demand and respecting technical constraints. Performance was compared against a baseline scheduler and assessed through feasibility checks and analysis of control actions.

For each of the three time series (electricity, heat, PV), three model classes were compared: SARIMA (seasonal ARIMA), LSTM (recurrent neural networks), and XGBoost (gradient boosting on lag features). For each model type, hyperparameters and structural choices were tuned (e.g., SARIMA orders and seasonal orders; LSTM hidden size, layers, epochs, and early stopping; XGBoost number of lags, tree depth, and number of estimators). Performance was evaluated using test RMSE, residual diagnostics (e.g., normality, bias), and, where applicable, training vs validation curves (loss or RMSE vs epochs or number of lags). Based on these comparisons, XGBoost was selected as the best-performing forecast model for all three variables and used to supply forecasts to the schedulers.

A baseline scheduler was run with historical data and again with forecast data. A 24-hour look-ahead linear program (LP) and a 24-hour reinforcement learning (RL) scheduler were implemented and evaluated both before and

after feasibility checks. Feasibility checks verified that solutions satisfied demand and technical limits (heat demand met, electricity demand met, boiler/CHP/buffer/battery ratings and coupling constraints). Total costs (gas, electricity, switch) were computed for each configuration. Control-variable trajectories (boiler input, CHP input, heat buffer discharge, battery discharge) were plotted for LP, RL, and the baseline to compare operational policies.

XGBoost consistently achieved the lowest test RMSE for electricity, heat, and PV. SARIMA provided interpretable seasonal structure (e.g., daily seasonality with period 24) and low RMSE but exhibited non-normal or skewed residuals (e.g., right-skewed for heat, zero-inflated and overpredicting for PV), which matters for prediction intervals. LSTMs showed fast convergence and no severe overfitting when kept small (e.g., 16 hidden units, 2 layers) with early stopping, but did not outperform XGBoost on test RMSE. The effect of lag features was examined for XGBoost; validation RMSE typically improved with more lags up to a plateau (e.g., around 12–14 lags for heat and PV), and shallower trees (e.g., max depth 3) generalized better than deeper ones. These results support the choice of XGBoost for all three forecasts.

The 24-hour LP before feasibility checks achieved the lowest total cost among all configurations, with reduced electricity and gas use and much lower switching cost than the baseline. Using forecast data in the baseline already reduced total cost relative to the historical-data baseline. Enforcing feasibility (e.g., ensuring heat demand is always met) increased the cost for both LP and RL: the LP became infeasible at one timestep (2024-12-09 10:00) because heat demand (193) exceeded what the boiler and CHP could jointly supply; after feasibility handling, the LP total cost rose. The RL scheduler showed very low switching cost before feasibility checks, but the

highest total cost after feasibility, with large increases in electricity and switching cost, highlighting that feasibility handling and the definition of “demand met” in the RL setting need careful interpretation.

Comparison of control variables (LP vs RL vs baseline) showed that the lower-cost schedulers (LP and RL) use dynamic, state-dependent policies rather than constant “always on” operation. Both turn the boiler off when not needed and operate the CHP strategically (on/off or reduced output) rather than at constant maximum output, as in the baseline. The heat buffer is used sparingly, in short bursts, by the LP; more continuously by the RL to smooth supply when CHP is off. Battery discharge is minimal across all schedulers, suggesting that in this setup, the battery’s main role is to charge (absorbing excess generation or cheap electricity) rather than to discharge frequently. Insights for “best” policies include: turn CHP and boilers on or up only when economically or operationally justified; use the heat buffer to decouple generation from demand; adapt decisions to forecasts and prices rather than relying on static rules.

Further investigations can be conducted on dynamic pricing, investigating more RL frameworks such as soft actor critic frameworks, more forecasting pipelines such as gated recurrent unit frameworks and Temporal fusion transformer frameworks, and on including metrics beyond cost, such as emissions reductions (e.g., CO_2) and other byproducts of energy systems.

Bibliography

[1] X. Wang, Y. Liu, C. Liu, and J. Liu, “Coordinating energy management for multiple energy hubs: From a transaction perspective,” *International Journal of Electrical Power & Energy Systems*, vol. 121, p. 106060, 2020.

[2] S. Stavrev and D. Ginchev, “Reinforcement learning techniques in optimizing energy systems,” *Electronics*, vol. 13, no. 8, p. 1459, 2024.

[3] W. Ko, J.-K. Park, M.-K. Kim, and J.-H. Heo, “A multi-energy system expansion planning method using a linearized load-energy curve: A case study in South Korea,” *Energies*, vol. 10, no. 10, p. 1663, 2017.

[4] L. Urbanucci, “Limits and potentials of Mixed Integer Linear Programming methods for optimization of polygeneration energy systems,” *Energy Procedia*, vol. 148, pp. 1199–1205, 2018.

[5] X. Fang, P. Hong, S. He, Y. Zhang, and D. Tan, “Multi-layer energy management and strategy learning for microgrids: A proximal policy optimization approach,” *Energies*, vol. 17, no. 16, p. 3990, 2024.

[6] Ö. Ö. Bozkurt, G. Biricik, and Z. C. Tayşi, “Artificial neural network and SARIMA based models for power load forecasting in Turkish electricity market,” *PloS one*, vol. 12, no. 4, p. e175915, 2017.

[7] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.

[8] S. Zhou, L. Zhou, M. Mao, H.-M. Tai, and Y. Wan, “An optimized heterogeneous structure LSTM network for electricity price forecasting,” *Ieee Access*, vol. 7, pp. 108161–108173, 2019.

[9] A. Mystakidis, P. Koukaras, N. Tsalikidis, D. Ioannidis, and C. Tjortjis, “Energy forecasting: A comprehensive review of techniques and technologies,” *Energies*, vol. 17, no. 7, p. 1662, 2024.